Modern Storages and Data Warehousing
Week 5 - File formats

Попов Илья, i.popov@hse.ru

1 - В предыдущих сериях

Recap прошлых занятий

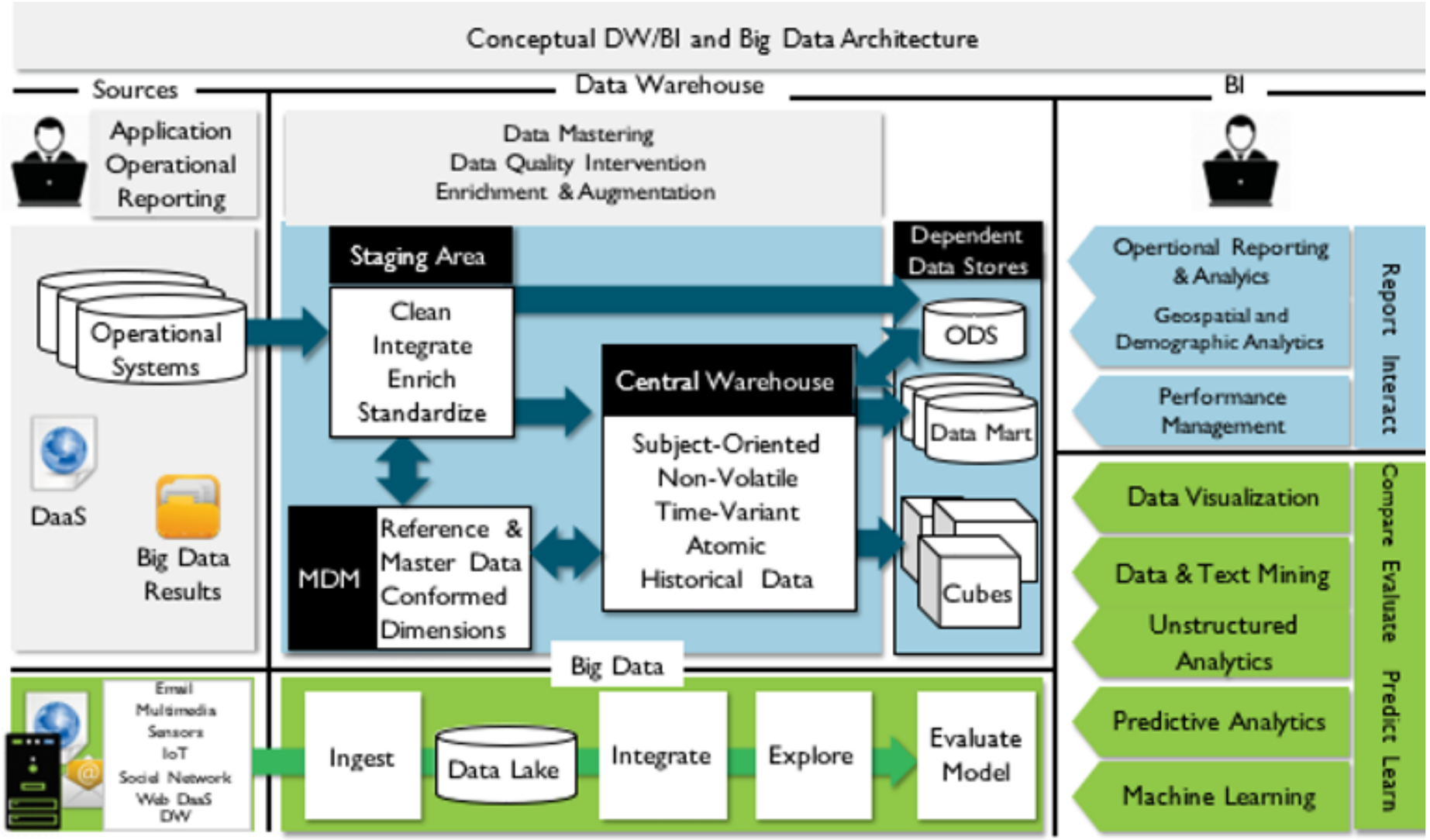


Figure 5: Date Warehouse Concept

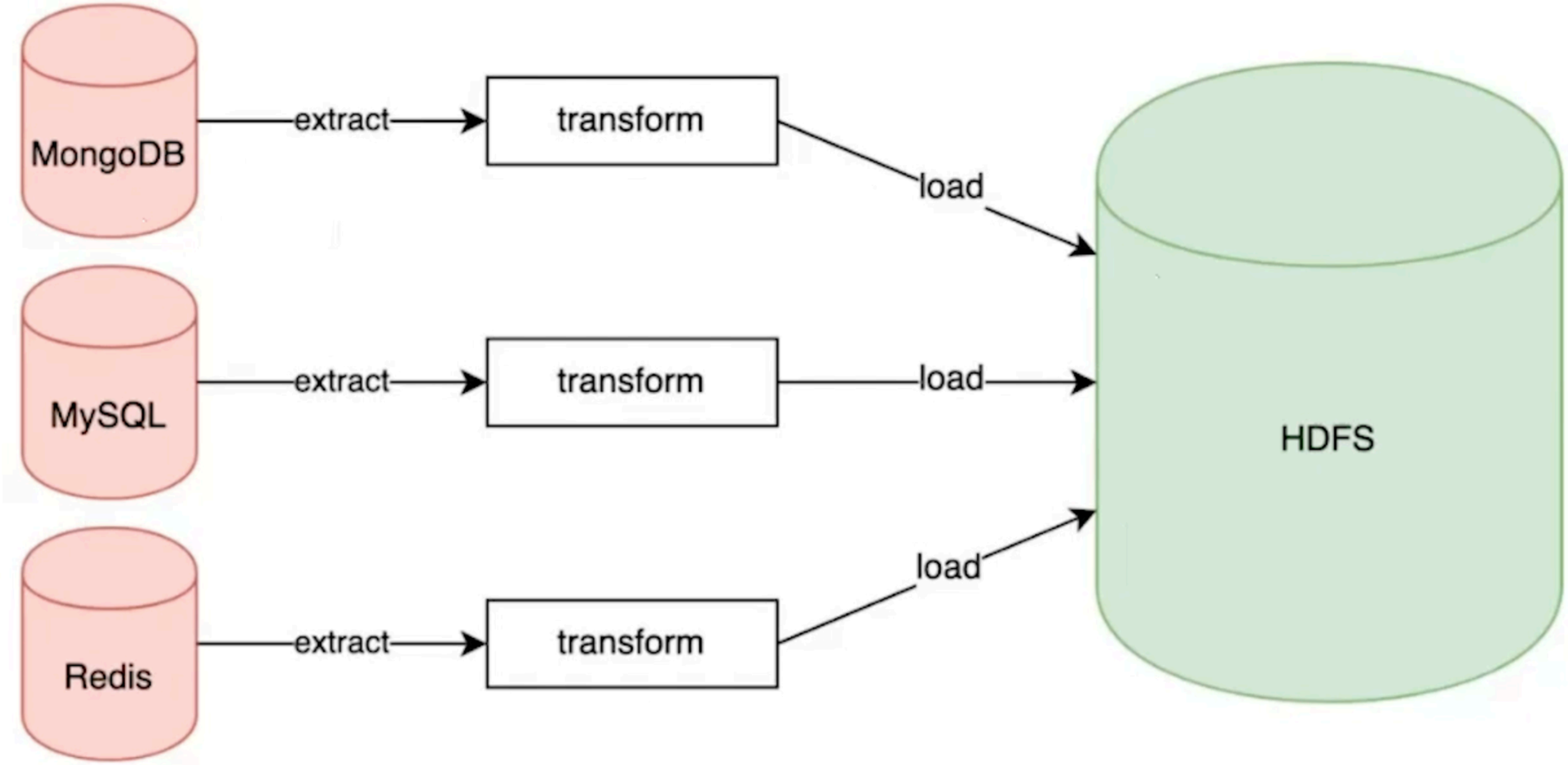
Recap прошлых занятий

〉 Мы знаем, где рождаются данные (**OLTP**), почему проблемы **нельзя решить только масштабированием** и как его не убить при работе (**репликация**)

〉 Мы знаем, куда сложить данные надолго и дешево (**S3**)

〉 Мы знаем, куда сложить большой объем данных для последующей обработки (**HDFS**), как работает **Hadoop**-экосистема и как нам потом все-таки эти данные обработать (**MapReduce**, **Spark**)

То, что мы знаем сейчас, выглядит примерно так



Recap прошлых занятий

〉 Мы знаем, где рождаются данные (**OLTP**), почему проблемы **нельзя решить только масштабированием** и как его не убить при работе (**репликация**)

〉 Мы знаем, куда сложить данные надолго и дешево (**S3**)

〉 Мы знаем, куда сложить большой объем данных для последующей обработки (**HDFS**), как работает **Hadoop**-экосистема и как нам потом все-таки эти данные обработать (**MapReduce**, **Spark**)

Проблема:

〉 Эту штуку мы хотим ускорить

〉 Данные с прода можем возить не только мы в удобном виде, но и внешние подрядчики в неудобном виде

2 - Форматы хранения данных

Самый простой кейс

```
<?xml version="1.0"?>
<catalog>
    <book id="bk101">
        <author>Gambardella, Matthew</author>
        <title>XML Developer's Guide</title>
        <genre>Computer</genre>
        <price>44.95</price>
        <publish_date>2000-10-01</publish_date>
        <description>An in-depth look at creating applications
            with XML.</description>
    </book>
    <book id="bk102">
        <author>Ralls, Kim</author>
        <title>Midnight Rain</title>
        <genre>Fantasy</genre>
        <price>5.95</price>
        <publish_date>2000-12-16</publish_date>
        <description>A former architect battles corporate zombies,
            an evil sorceress, and her own childhood to become queen
            of the world.</description>
    </book>
    ....
```

```
spark.read
    .option("rowTag", "book")
    .option("rootTag", "books")
    .format("com.databricks.spark.xml")
    .load(path)
    .withColumn("price", cast("price"))
    .agg(f.max("price"))
    .show()
```

Самый простой кейс

```
<?xml version="1.0"?>
<catalog>
    <book id="bk101">
        <author>Gambardella, Matthew</author>
        <title>XML Developer's Guide</title>
        <genre>Computer</genre>
        <price>44.95</price>
        <publish_date>2000-10-01</publish_date>
        <description>An in-depth look at creating applications
            with XML.</description>
    </book>
    <book id="bk102">
        <author>Ralls, Kim</author>
        <title>Midnight Rain</title>
        <genre>Fantasy</genre>
        <price>5.95</price>
        <publish_date>2000-12-16</publish_date>
        <description>A former architect battles corporate zombies,
            an evil sorceress, and her own childhood to become queen
            of the world.</description>
    </book>
    ....
```

```
spark.read
    .option("rowTag", "book")
    .option("rootTag", "books")
    .format("com.databricks.spark.xml")
    .load(path)
    .withColumn("price", cast("price"))
    .agg(f.max("price"))
    .show()
```

Мысли?
Бро/не бро?
Будем так делать?

Какие форматы бывают вообще?

〉 Специфичные для языка:
python — Pickle
Java — Serializable, Kryo
Ruby — Marshal

Какие форматы бывают вообще?

〉Специфичные для языка:
python — Pickle
Java — Serializable, Kryo
Ruby — Marshal

Мысли?
Бро/не бро?
Будем так делать?

Какие форматы бывают вообще?

〉 Специфичные для языка:
python — Pickle
Java — Serializable, Kryo
Ruby — Marshal

〉 Нельзя десериализовать в другом языке
〉 Плохо с версионированием
〉 Плохо с производительностью
〉 Небезопасно: десериализация недоверенных данных = выполнение кода

Какие форматы бывают вообще?

〉 Текстовые форматы:
JSON
XML
CSV
Yaml

Какие форматы бывают вообще?

〉 Текстовые форматы:
JSON
XML
CSV
Yaml

Мысли?
Бро/не бро?
Будем так делать?

Какие форматы бывают вообще?

- Текстовые форматы:
  JSON
  XML
  CSV
  Yaml

- Проблемы с типами данных (различия в записи int/long/float/string)
- Проблемы с экранированием
- Избыточность
- Неэффективное представление

Какие форматы бывают вообще?

〉 Текстовые форматы:
JSON
XML
CSV
Yaml

〉 Решение проблемы - **давайте просто рядом положим схему данных**
¯_(ツ)_/¯

Что такое схема данных?

```
{
    "type": "record",
    "namespace": "com.example",
    "name": "Person",
    "fields": [
        { "name": "firstName", "type": "string" },
        { "name": "lastName", "type": "string" },
        { "name": "age", "type" : "int"}
    ]
}
```

Что такое схема данных?

```
{
    "type": "record",
    "namespace": "com.example",
    "name": "Person",
    "fields": [
        { "name": "firstName", "type": "string" },
        { "name": "lastName", "type": "string" },
        { "name": "age", "type" : "int"}
    ]
}
```

```
{
    "type": "record",
    "namespace": "com.example",
    "name": "Person",
    "fields": [
        { "name": "firstName", "type": "string" },
        { "name": "lastName", "type": "string" },
        { "name": "age", "type" : "int"},
        { "name": "interests",   "type": {"type": "array", "items": "string"}},
        { "name": "address", "type" : {
                    "type" : "record",
                    "name": "mailing_address",
                    "fields": [
                      {"name" : "street", "type" : "string"},
                      {"name" : "city", "type" : "string"},
                      {"name" : "country",  "type" : "string",  "default" : "NONE"},
                      {"name" : "zip",  "type" : "string",  "default" : "NONE"}
                    ]},
                    "default" : {}
            }
    ]
}
```

Эволюция схемы

〉 Со временем мы захотим менять/изменять формат, как наши данные хранятся

〉 Эволюция схемы - это процесс её изменения во времени

〉 О чем нужно думать:

Обратная совместимость

Прямая совместимость

Эволюция схемы

〉 Со временем мы захотим менять/изменять формат, как наши данные хранятся

〉 Эволюция схемы - это процесс её изменения во времени

〉 О чем нужно думать:

Обратная совместимость - новый код может прочитать старую схему (новый код читает **данные, записанные старым кодом**)

Прямая совместимость - старый код может прочитать новую схему (старый код читает данные, записанные новым кодом)

Эволюция схемы

〉 Со временем мы захотим менять/изменять формат, как наши данные хранятся

〉 Эволюция схемы - это процесс её изменения во времени

〉 О чем нужно думать:

Обратная совместимость - новый код может прочитать старую схему

Прямая совместимость - старый код может прочитать новую схему

〉 В разных форматах совместимость реализуется по-разному

Apache Avro

〉 Релиз в 2009 как часть Hadoop-экосистемы
〉 Поддержка Spark из коробки
〉 Библиотеки для сериализации/десериализации на *практически* любом языке
〉 Использует схему данных и прямую/обратную совместимость

Пример

〉 Пример для MessagePack - бинарный формат сериализации с моделью данных JSON

〉 Сериализованное сообщение:

```
{
  “userName”: “Martin”,
  “favoriteNumber”: 1337,
  “interests”: [
    “daydreaming”, “hacking”
  ]
}
```

〉 Занимает 66 байт

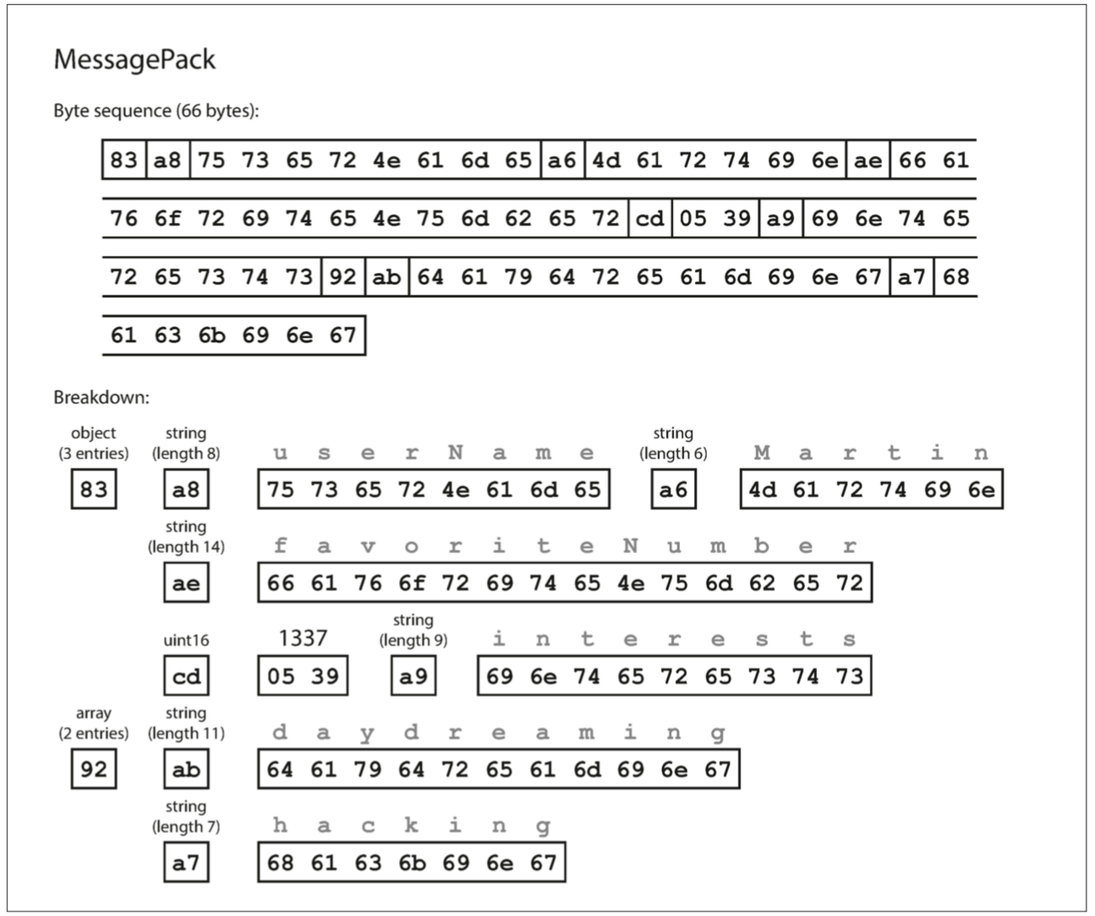


Figure 4-1. Example record (Example 4-1) encoded using MessagePack.

Пример

〉 Пример для Apache Avro

〉 Сериализованное сообщение:

```
{
  “userName”: “Martin”,
  “favoriteNumber”: 1337,
  “interests”: [
    “daydreaming”, “hacking”
  ]
}
```

〉 Занимает 32 байта

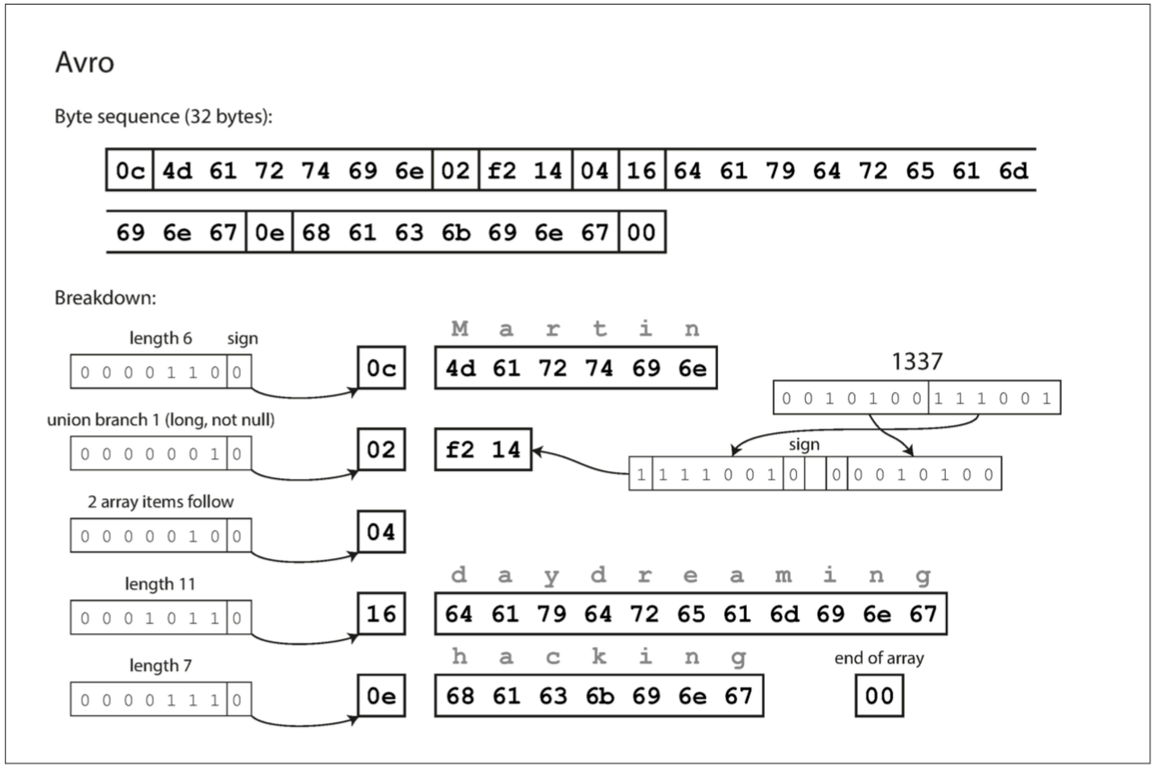


Figure 4-5. Example record encoded using Avro.

Apache Avro

- Схема хранится рядом с кодом в json-формате
- Схема хранится в данных в начале avro-файла
- Парсер смотрит на эти 2 схемы и понимает, как читать данные

- Можно удалять поля (если задано default-значение)
- Можно добавлять поля (если задано default-значение)
- Можно менять поля местами
- Можно менять тип данных, если типы конвертируются друг в друга
- Можно переименовывать поля (alliases)

Фиксировать схему данных - это хорошо

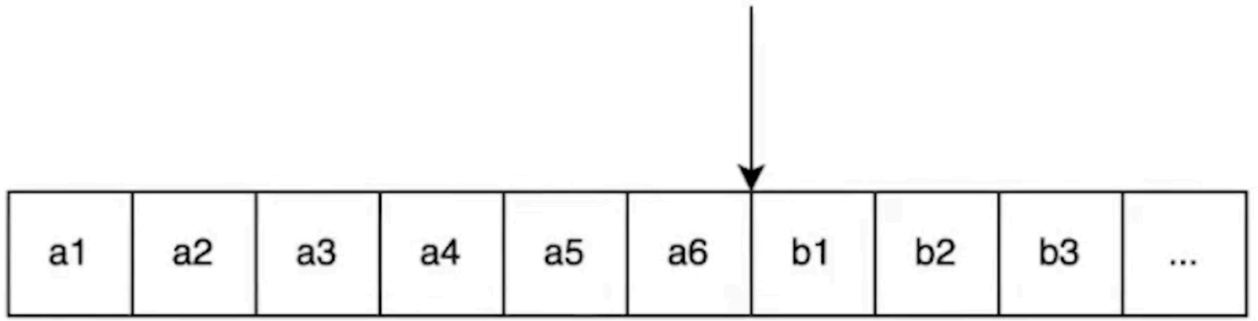
Построчные форматы

	a	b	c	d
1	a1	b1	c1	d1
2	a2	b2	c2	d2
3	a3	b3	c3	d3
4	a4	b4	c4	d4
5	a5	b5	c5	d5
6	a6	b6	c6	d6

a1	b1	c1	d1	a2	b2	c2	d2	...

Поколоночные форматы

	a	b	c	d
1	a1	b1	c1	d1
2	a2	b2	c2	d2
3	a3	b3	c3	d3
4	a4	b4	c4	d4
5	a5	b5	c5	d5
6	a6	b6	c6	d6



Поколоночные форматы

Плюсы:

〉 Можно прочитать только одну или несколько колонок

〉 Можно оптимизировать хранение - применить кодирование (dictionary, RLE, delta) + сжатие страниц кодеком (Snappy/ZSTD/GZIP)

〉 Можно использовать векторизированные вычисления

Поколоночные форматы

Плюсы:

〉Можно прочитать только одну или несколько колонок

〉Можно оптимизировать хранение - применить кодирование (dictionary, RLE, delta) + сжатие страниц кодеком (Snappy/ZSTD/GZIP)

〉Можно использовать векторизированные вычисления

Важно:

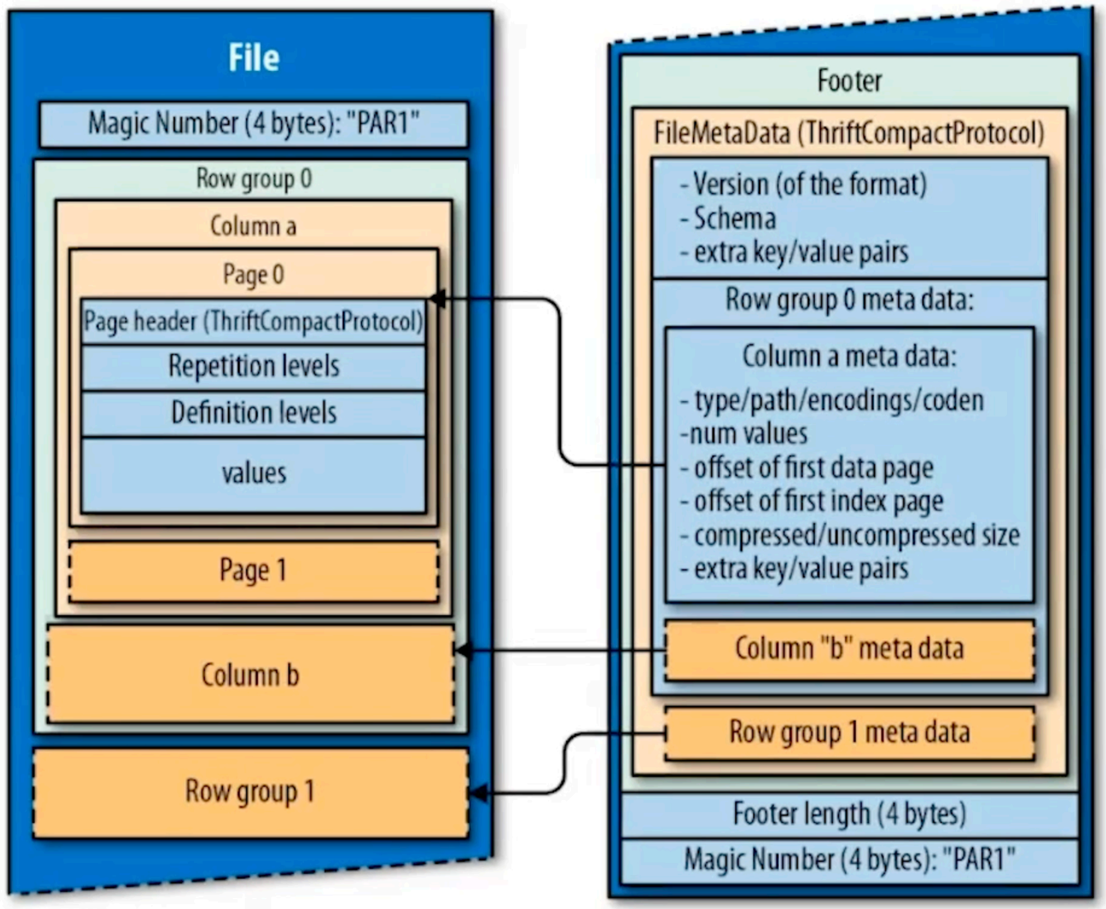
〉Эффективность сжатия зависит от сортировки

Apache Parquet

〉 Релиз в 2013, Twitter + Cloudera, по мотивам Google Dremel;
〉 Apache TLP с 2015
〉 Поддержка Spark из коробки
〉 Библиотеки для сериализации/десериализации на любом языке
〉 Схема в футере файла; ограниченная эволюция (добавление колонок, merge schema в движке);

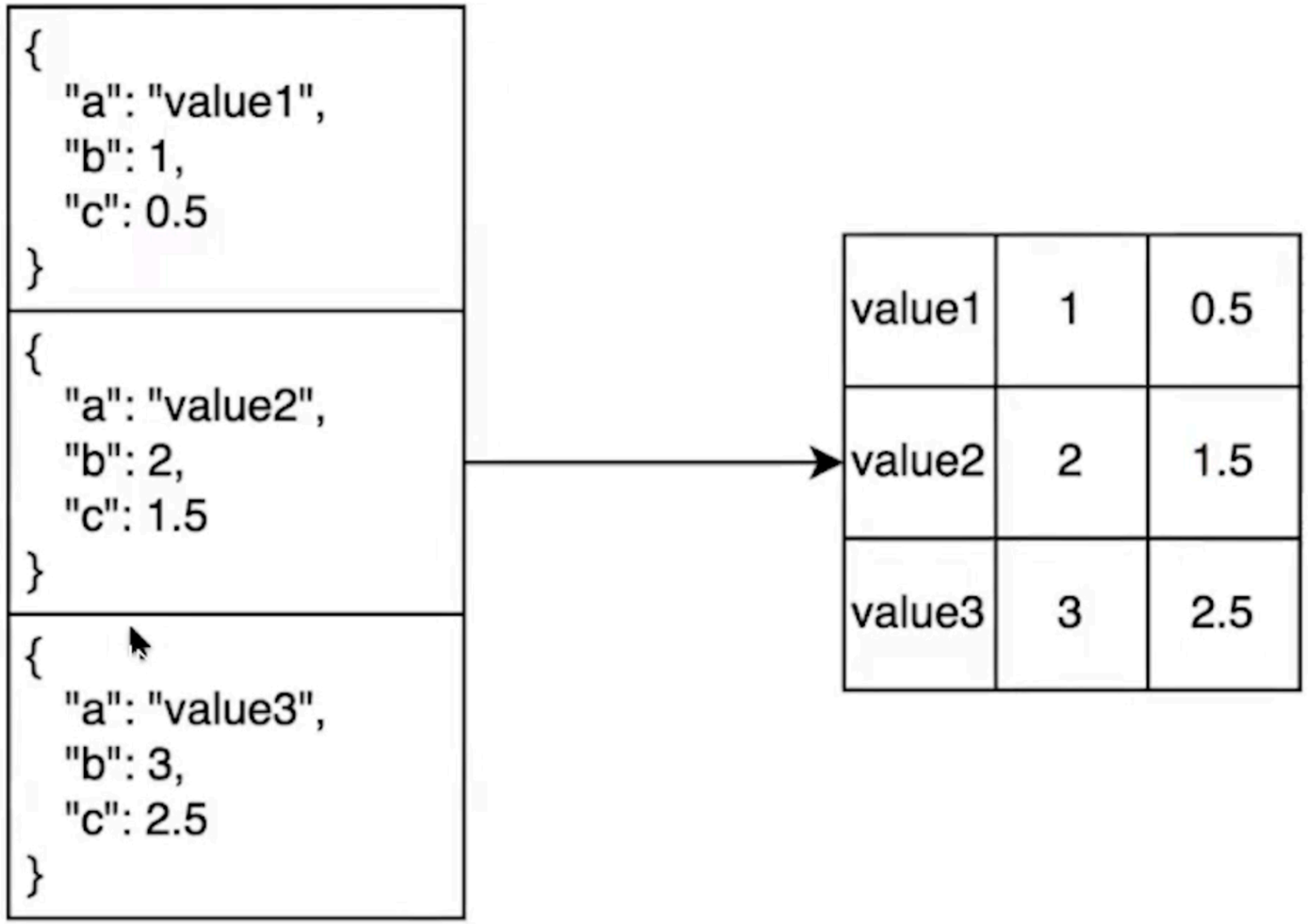
〉 Использует метаданные и predicate pushdown

Apache Parquet

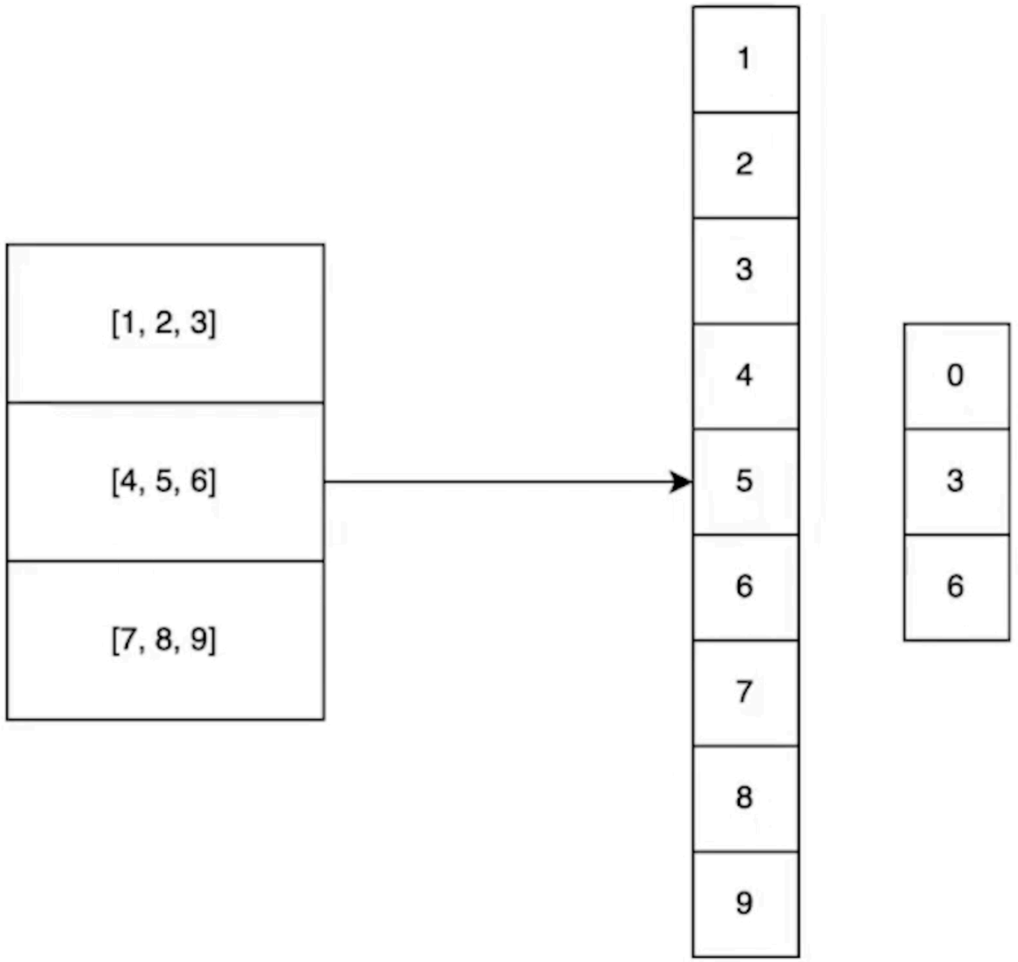


a1	b1	c1	d1
a2	b2	c2	d2
a3	b3	c3	d3
a4	b4	c4	d4
a5	b5	c5	d5
a6	b6	c6	d6

Сложные типы



Сложные типы



Колоночные форматы - это хорошо для Big Data

One more thing - Partition Discovery

```
path
└── to
    └── table
        ├── gender=male
        │   ├── ...
        │   │
        │   ├── country=US
        │   │   └── data.parquet
        │   ├── country=CN
        │   │   └── data.parquet
        │   └── ...
        └── gender=female
            ├── ...
            │
            ├── country=US
            │   └── data.parquet
            ├── country=CN
            │   └── data.parquet
            └── ...
```

```
root
|-- name: string (nullable = true)
|-- age: long (nullable = true)
|-- gender: string (nullable = true)
|-- country: string (nullable = true)
```

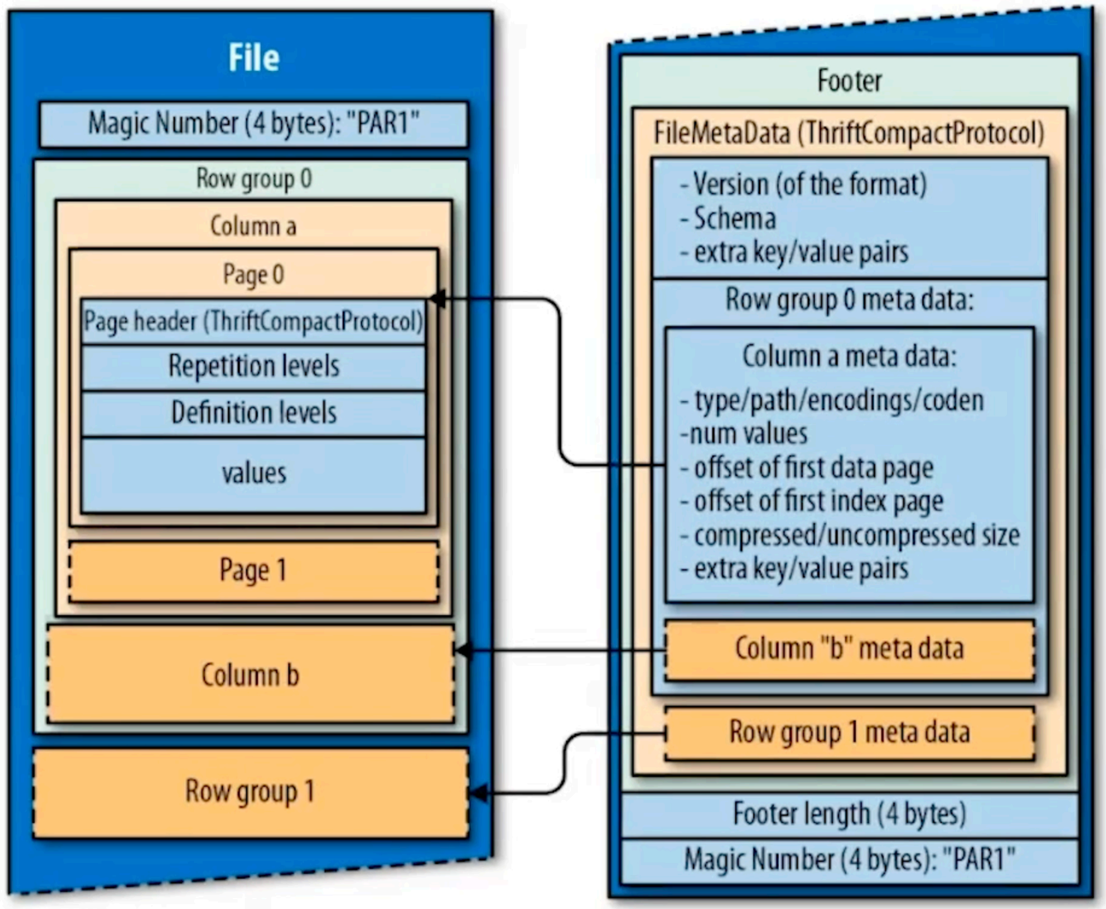
3 - Iceberg

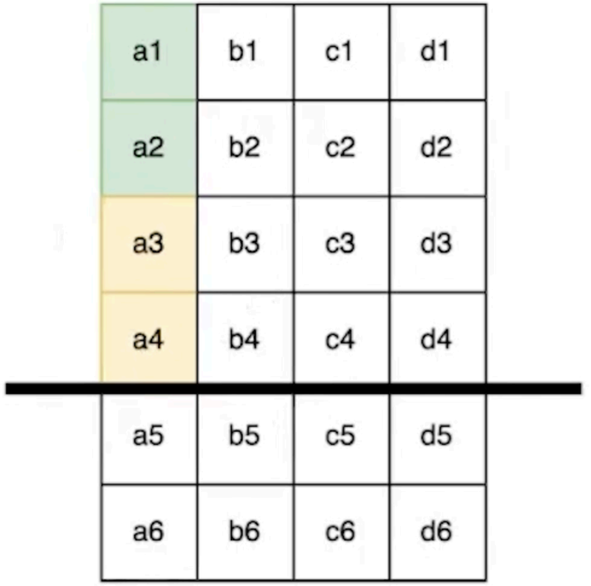
Partition Discovery

```
path
└── to
    └── table
        ├── gender=male
        │   ├── ...
        │   │
        │   ├── country=US
        │   │   └── data.parquet
        │   ├── country=CN
        │   │   └── data.parquet
        │   └── ...
        └── gender=female
            ├── ...
            │
            ├── country=US
            │   └── data.parquet
            ├── country=CN
            │   └── data.parquet
            └── ...
```

```
root
|-- name: string (nullable = true)
|-- age: long (nullable = true)
|-- gender: string (nullable = true)
|-- country: string (nullable = true)
```

Apache Parquet





Apache Parquet

- Для того, чтобы прочитать большой файл, нужно:
  - Получить список всех его патриций
  - Прочитать метаинформацию/статистики из этих партиций
- Если в момент чтения произойдет запись/изменение, то будет конфликт
- В некоторых видах хранилищ операция list может быть очень затратной (например, в S3)

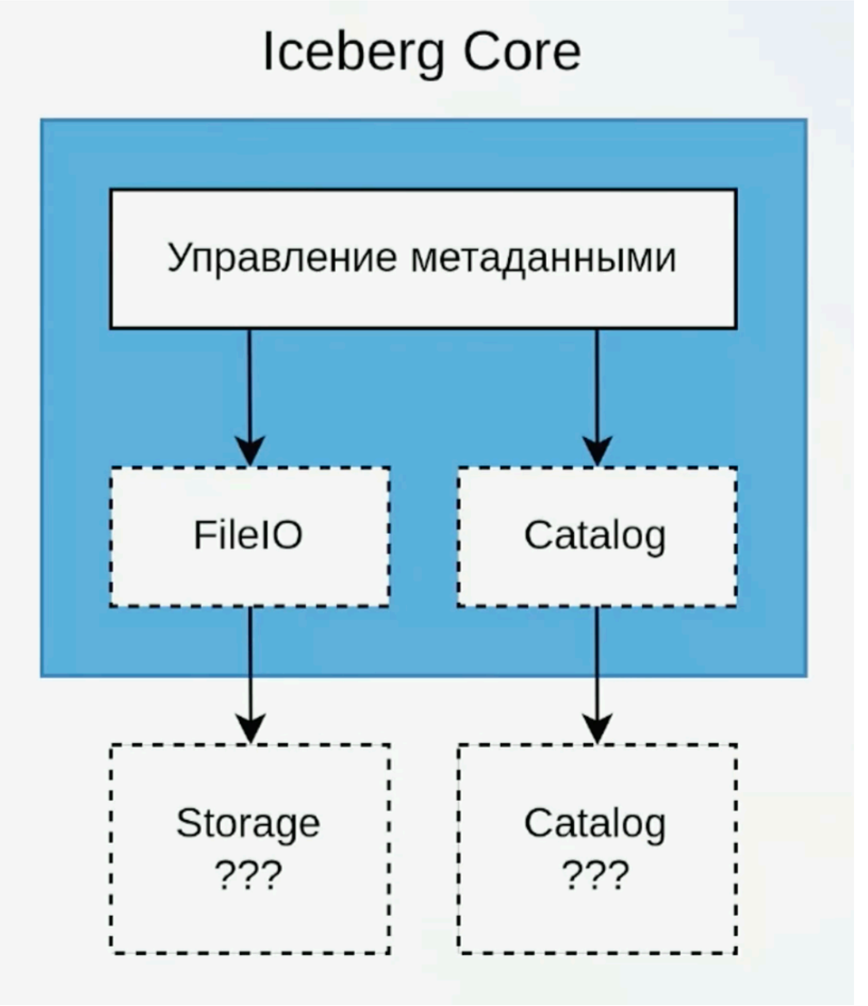
Apache Parquet

- Для того, чтобы прочитать большой файл, нужно:
  - Получить список всех его патриций
  - Прочитать метаинформацию/статистики из этих партиций
- Если в момент чтения произойдет запись/изменение, то будет конфликт
- В некоторых видах хранилищ операция list может быть очень затратной (например, в S3)

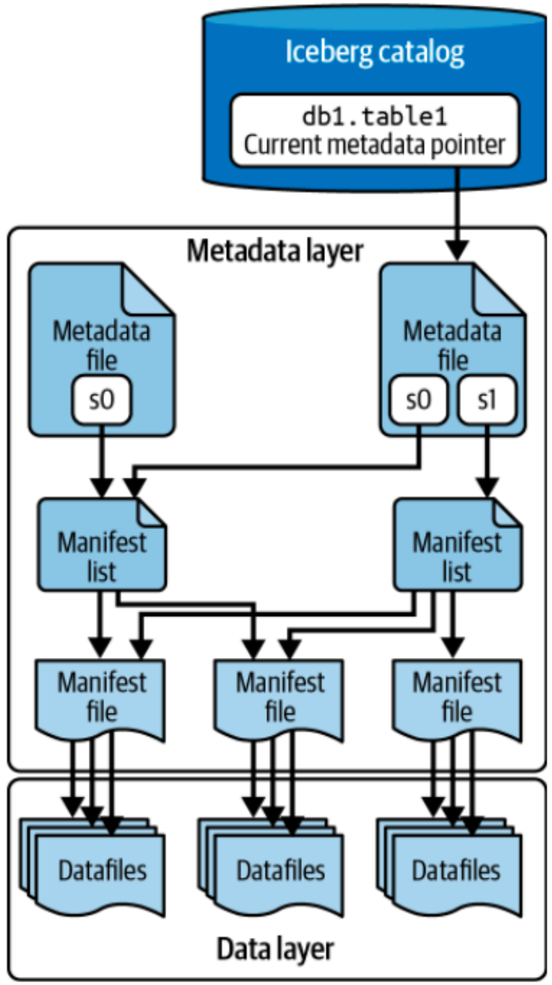
- Хотим:
  - Не делать кучу запросов/не нагружать систему для чтения метаданных
  - Иметь какое-нибудь подобие ACID-гарантий

Apache Iceberg

- Был разработан в 2017 году внутри Netflix
- В 2018 году стал Open Source и уехал в Apache Foundation
- На тренде “облачного DWH” обрел взрывную популярность
- Iceberg предоставляет:
  - API для управления метаданными (Iceberg Core)
  - FileIO для записи файлов и общения со Storage
  - Контракт для общения с Catalog



Как устроен Iceberg

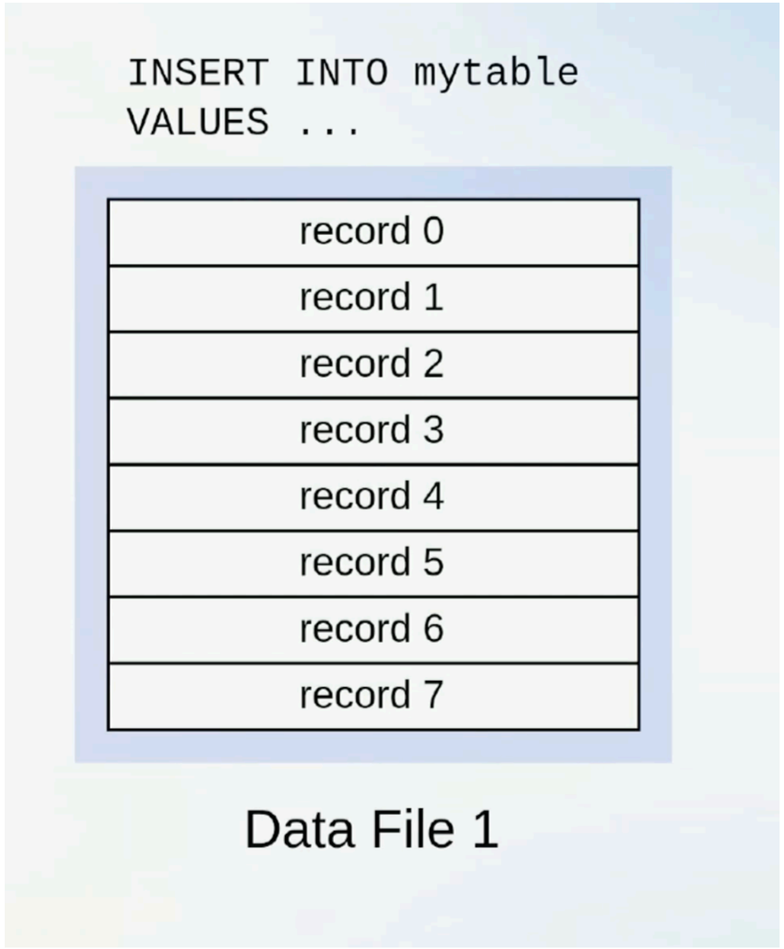


Metadata – ключевой файл метаданных в Apache Iceberg.

Все остальное – разного рода оптимизации:

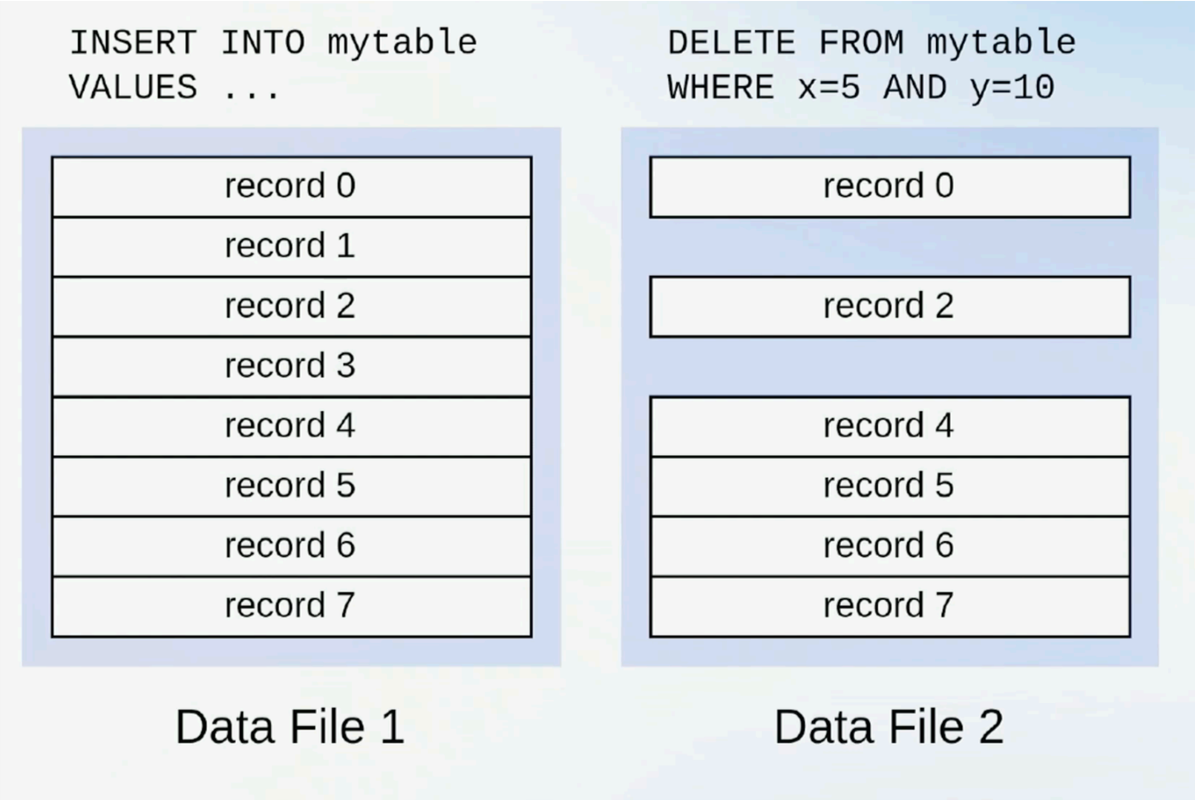
- Metadata file (JSON) - корень состояния таблицы
- Manifest list, Manifest file (Avro) - иерархический индекс файлов, чтобы не переписывать весь список при каждом коммите
- Статистики (Puffin, опционально) - расширенные статистики и deletion vectors

Что умеет Iceberg



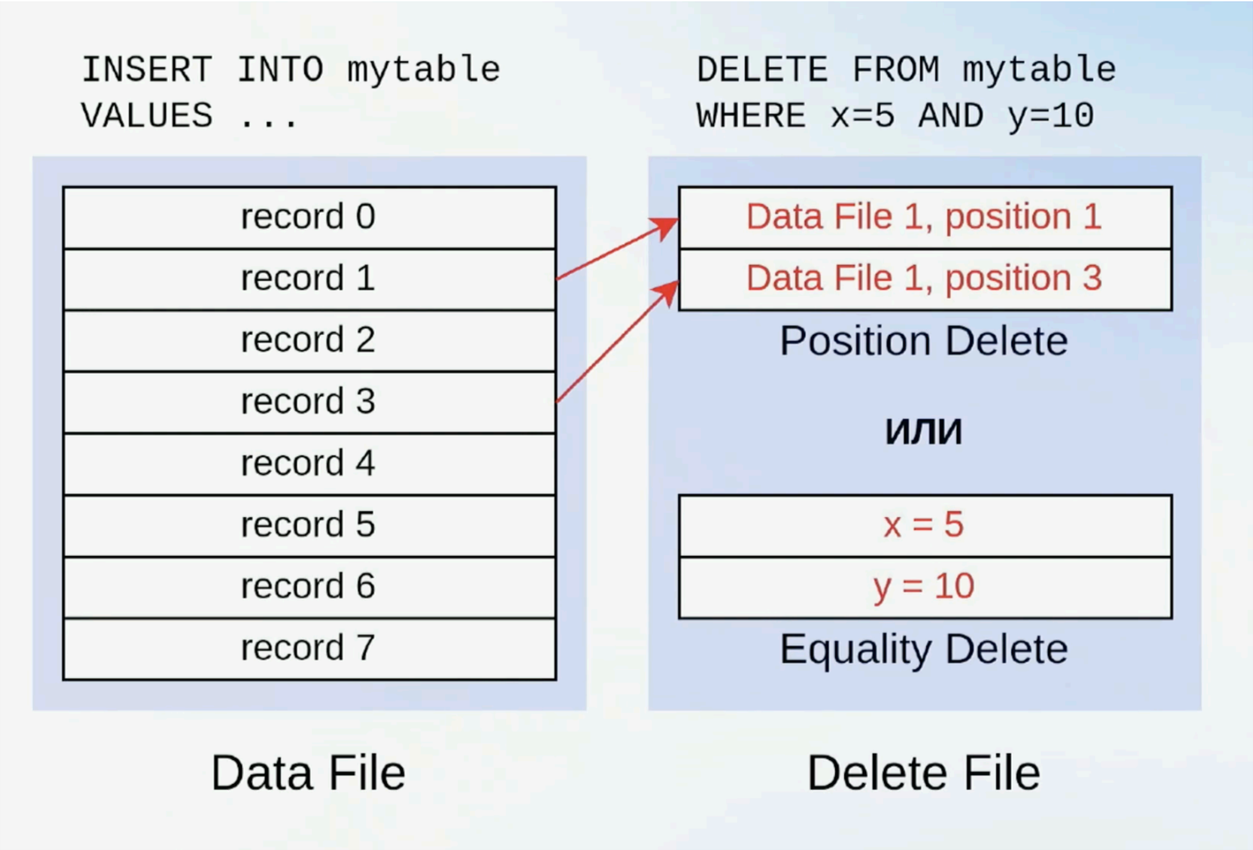
〉Хранит любой бинарь внутри - Avro, ORC, Parquet…

Что умеет Iceberg



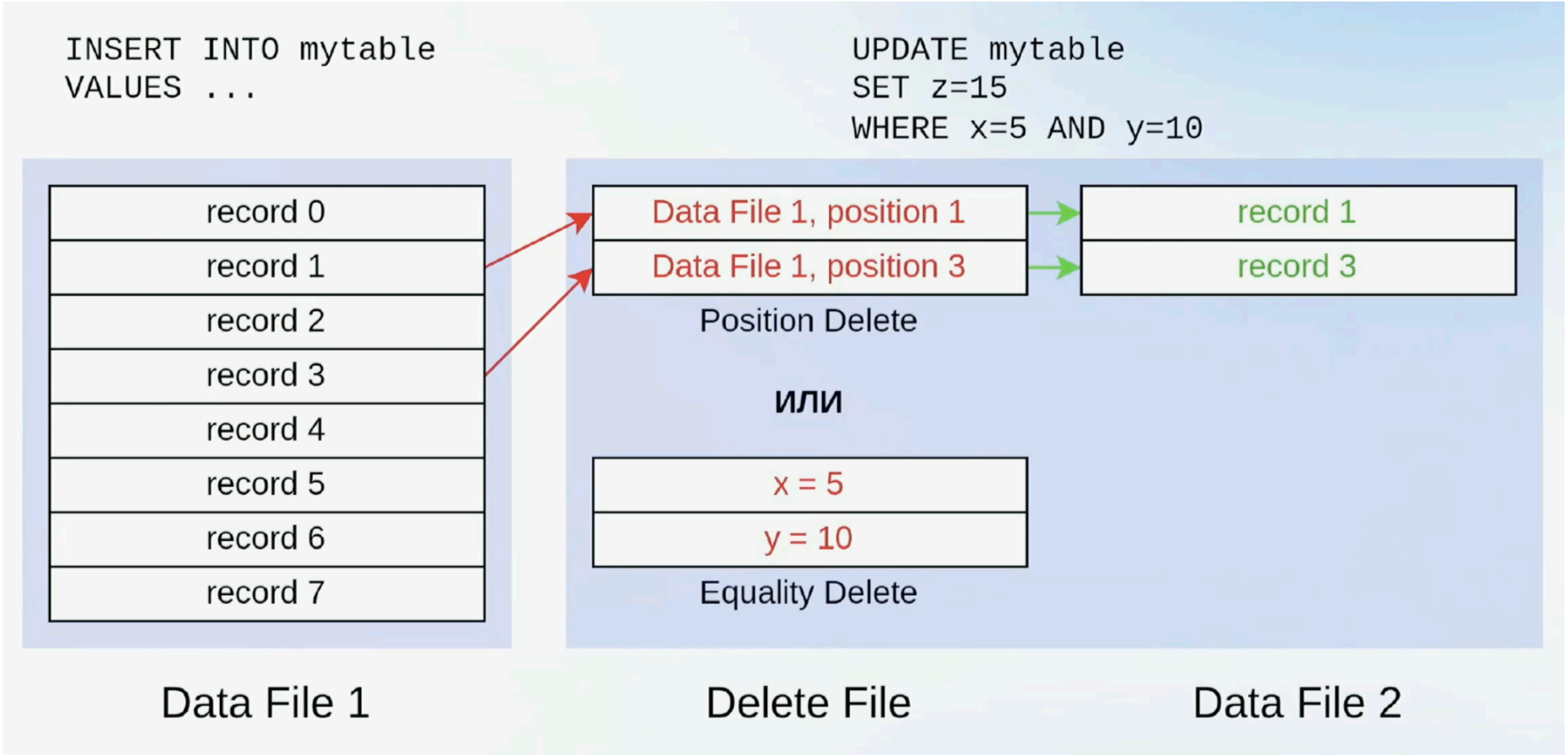
- Хранит любой бинарь внутри - Avro, ORC, Parquet…
- Умеет в Copy-on-write

Что умеет Iceberg

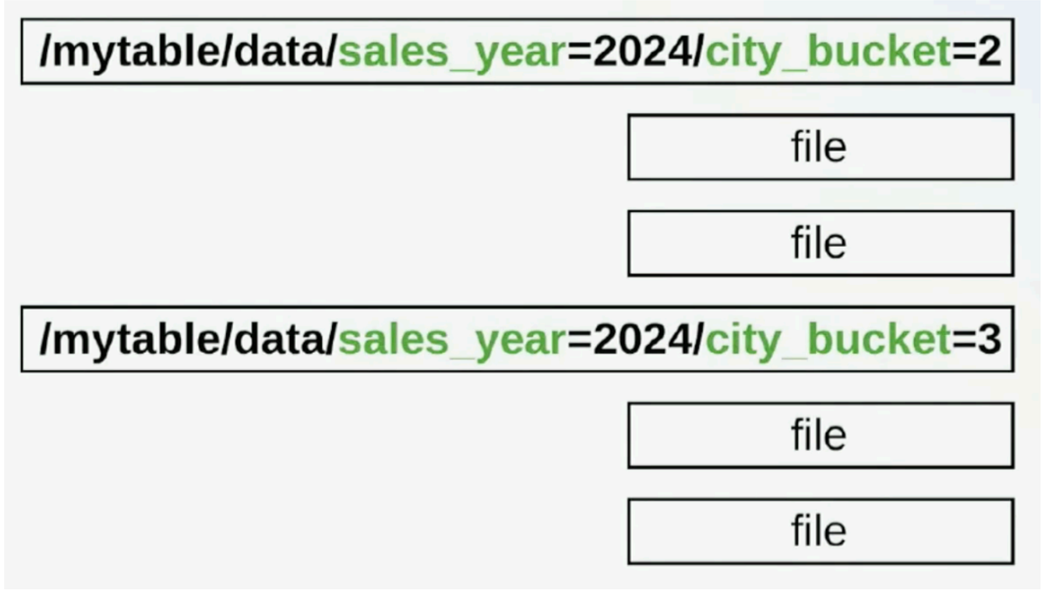


- Хранит любой бинарь внутри - Avro, ORC, Parquet…
- Умеет в Copy-on-write
- Умеет в Merge-on-read

Что умеет Iceberg



Что умеет Iceberg

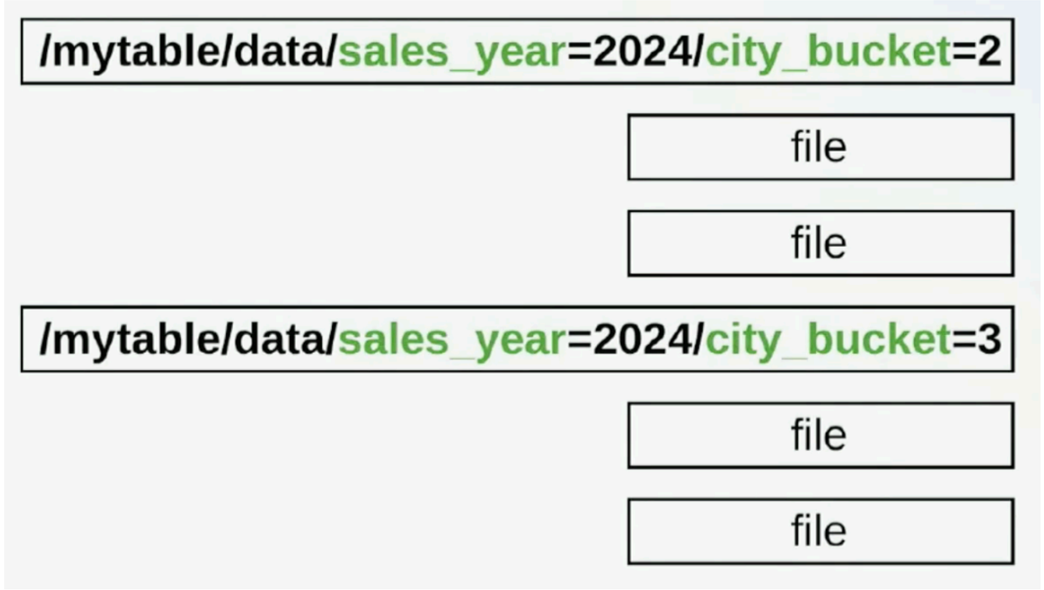



```
sales_year  = year(sales_date)

city_bucket = bucket(city, 16)
```

- Хранит любой бинарь внутри - Avro, ORC, Parquet…
- Умеет в Copy-on-write
- Умеет в Merge-on-read
- Умеет в виртуальные колонки для партицирования

Что умеет Iceberg

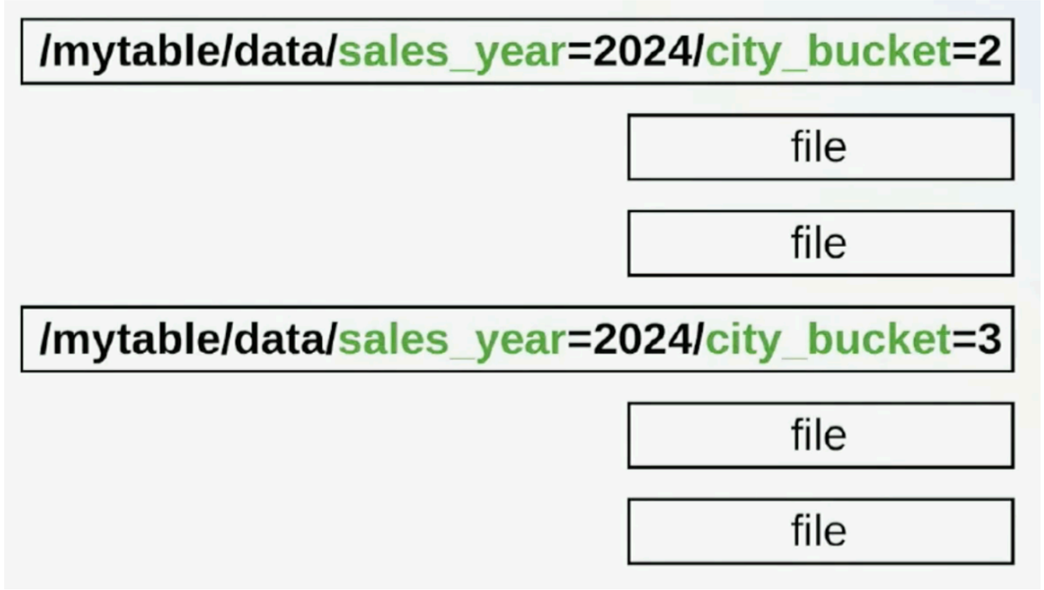



```
sales_year  = year(sales_date)

city_bucket = bucket(city, 16)
```

- Хранит любой бинарь внутри - Avro, ORC, Parquet…
- Умеет в Copy-on-write
- Умеет в Merge-on-read
- Умеет в виртуальные колонки для партицирования
- Собирает точные статистики для data skipping и приблизительные для оптимизаций (theta sketches)

Что умеет Iceberg




```
sales_year  = year(sales_date)

city_bucket = bucket(city, 16)
```

- Хранит любой бинарь внутри - Avro, ORC, Parquet, Arrow…
- Умеет в Copy-on-write
- Умеет в Merge-on-read
- Умеет в виртуальные колонки для партицирования
- Собирает точные статистики для data skipping и приблизительные для оптимизаций (theta sketches)
- Constraints - Identifier Fields и Sort Fields

Что еще умеет Iceberg

- Time Travel и Snapshot Rollback
- Schema evolution

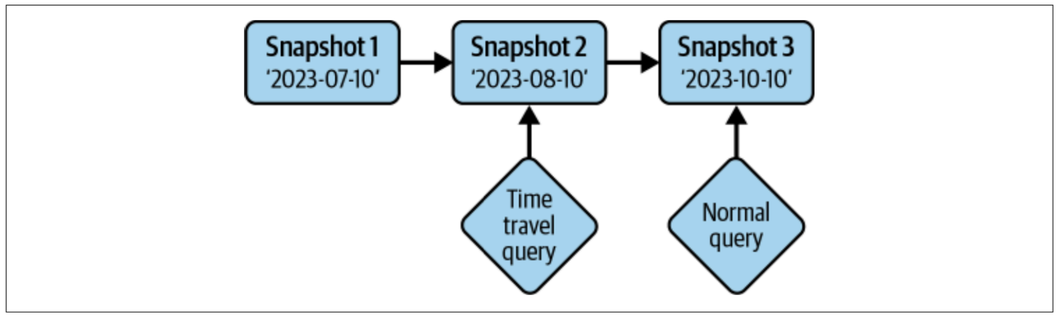


Figure 1-10. Querying the table as it was using time travel

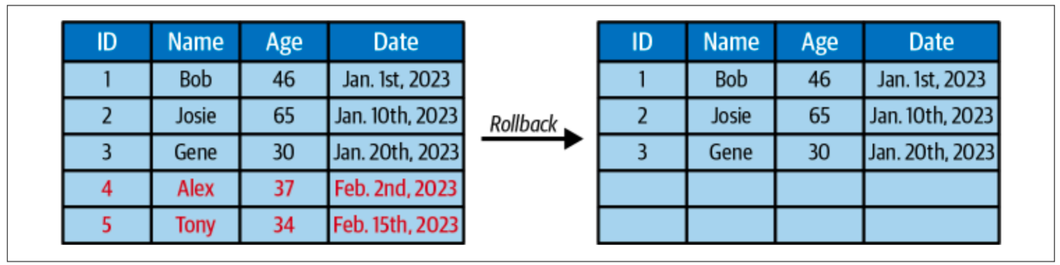


Figure 1-11. Moving the table's state to a previous point in time by rolling back

Как работает запись таблицы Iceberg

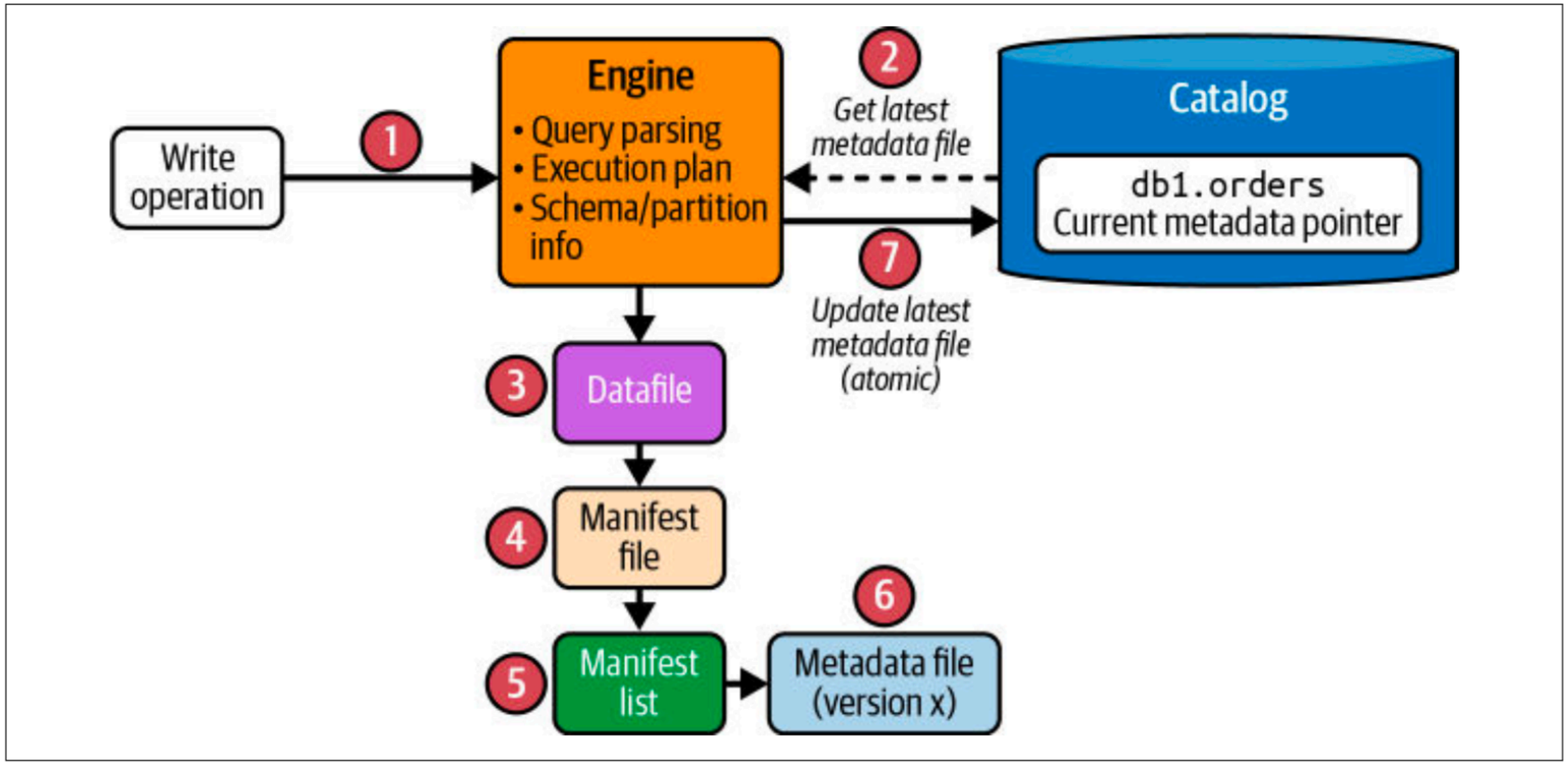


Figure 3-2. Overview of the Apache Iceberg write process

Как работает чтение таблицы Iceberg

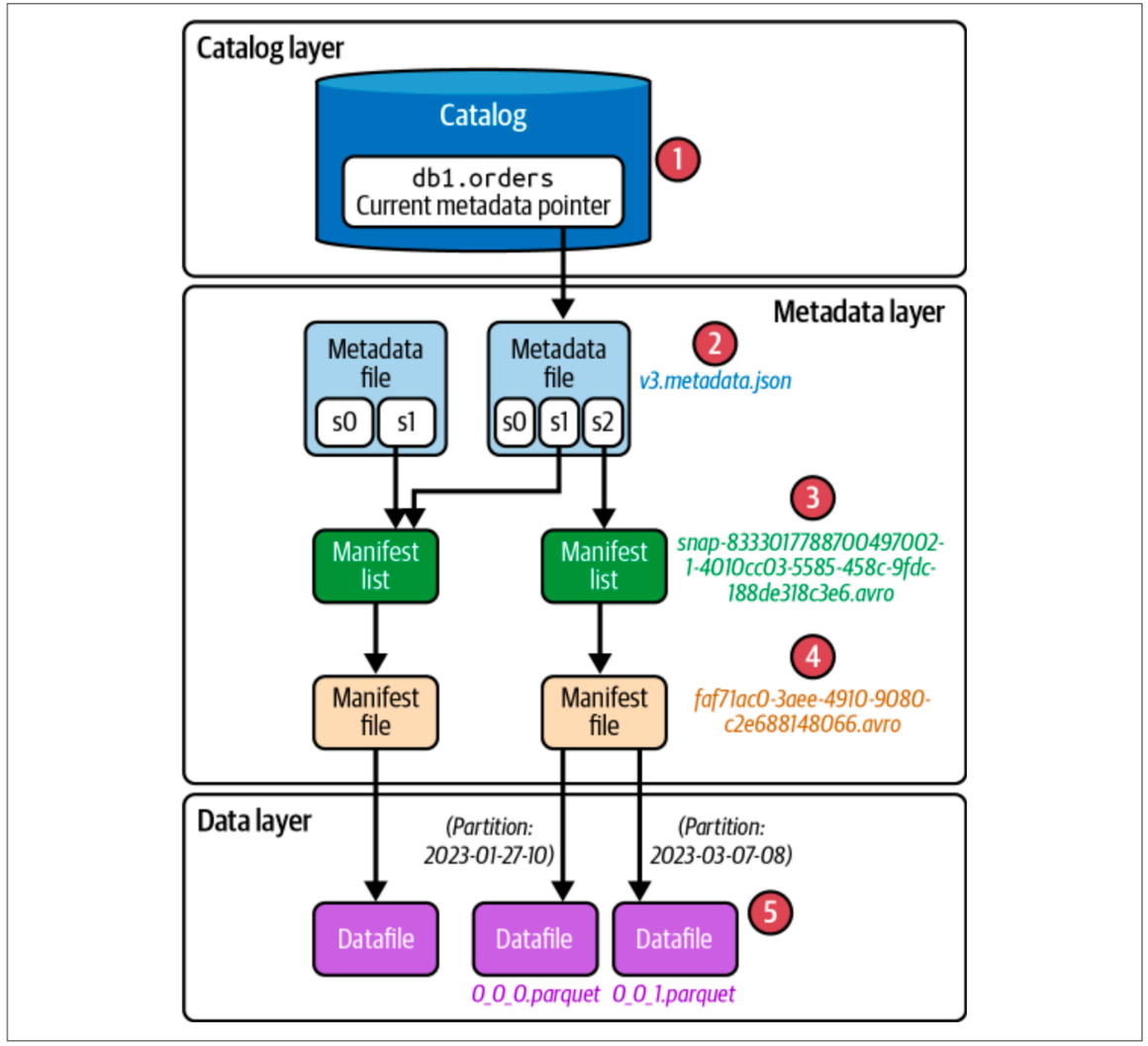
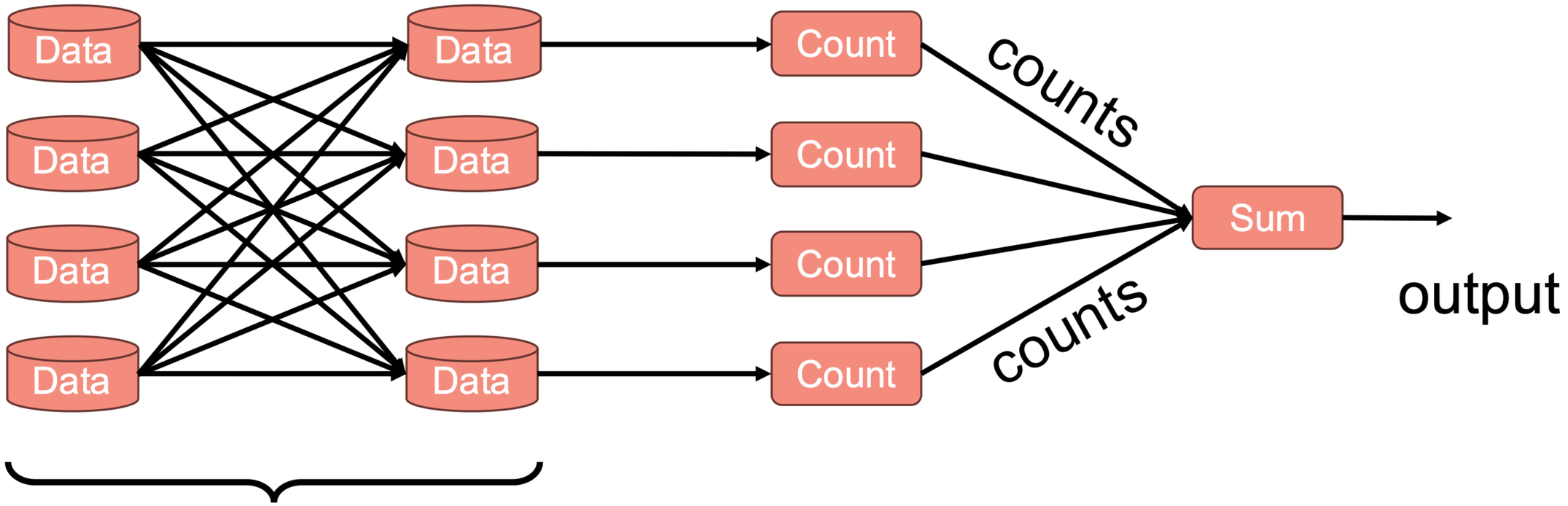


Figure 3-6. How a READ query works in Apache Iceberg

- Забрать из каталога текущий metadata файл
- Найти нужный snapshot
- Забрать его manifest list
- Забрать нужные manifest files
- Прочитать data files

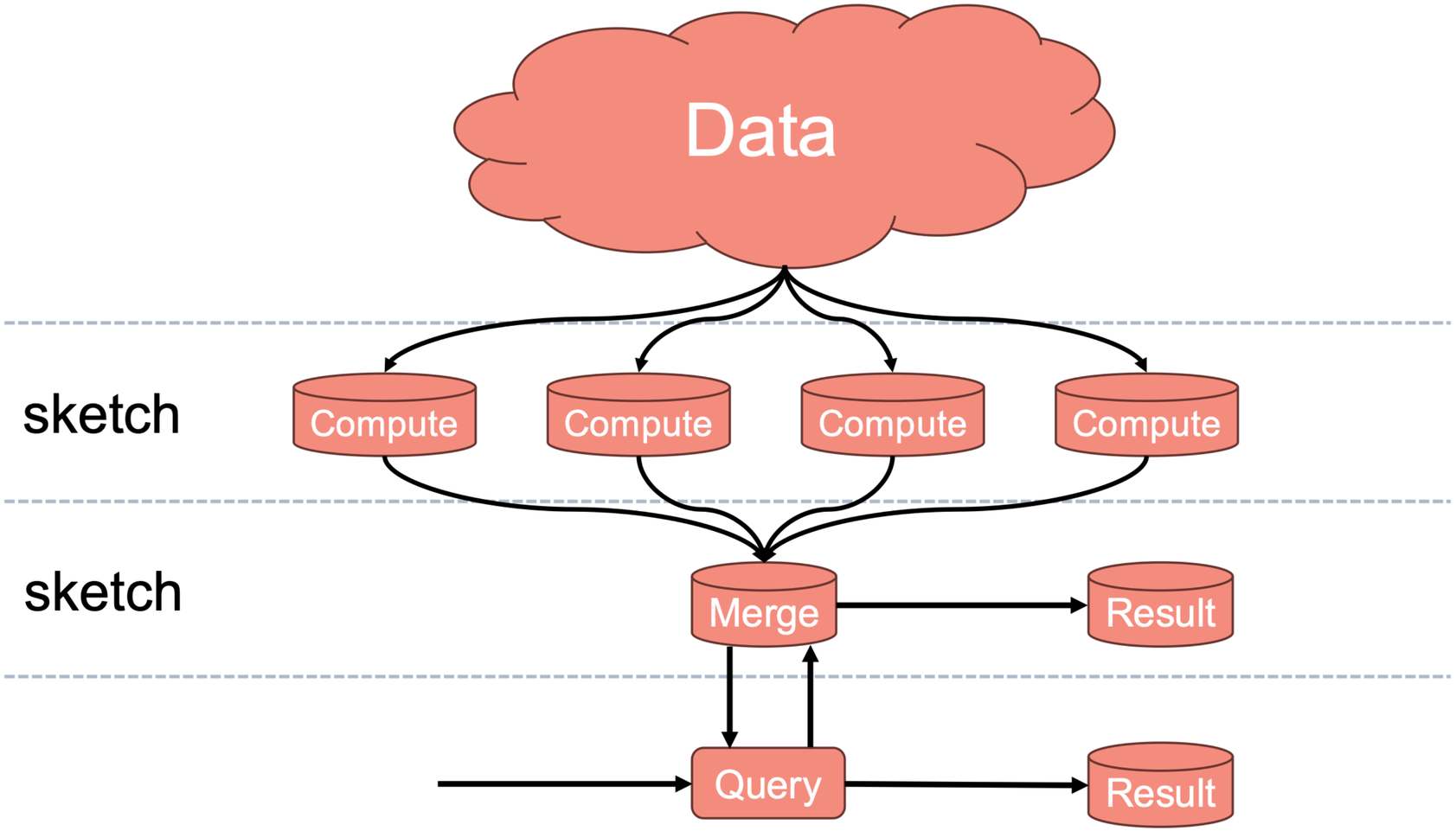
4 - Бонус

Data Sketches

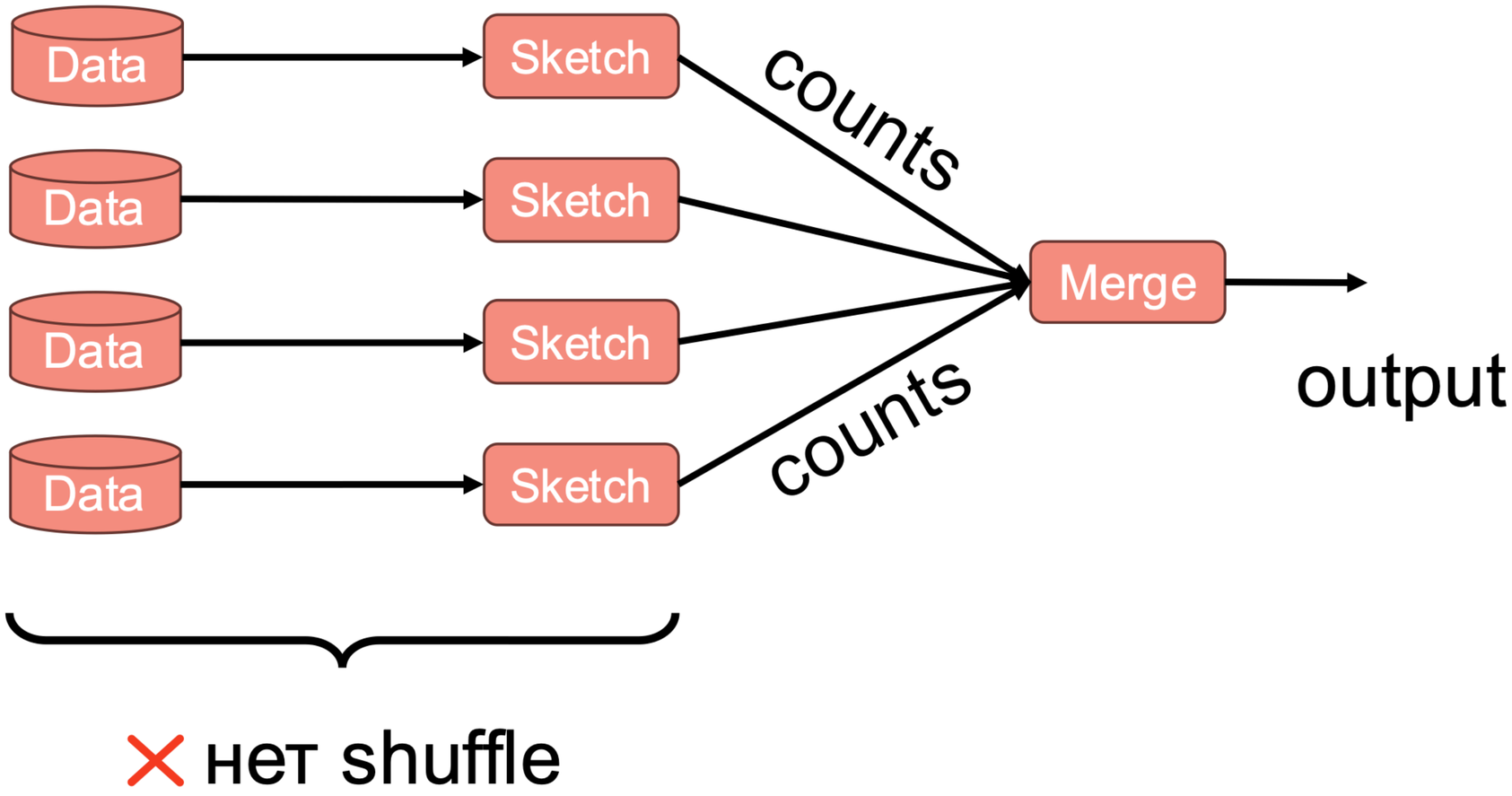


shuffle – дороro

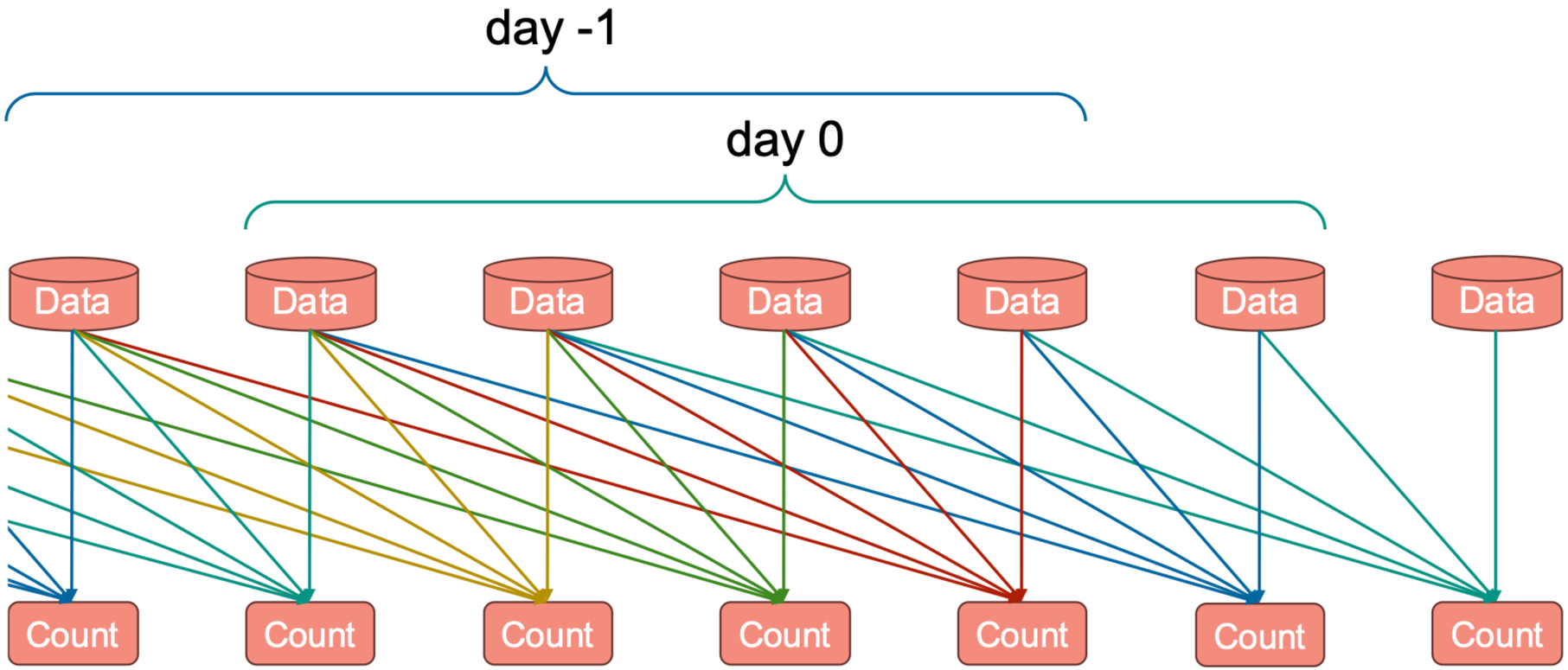
Data Sketches



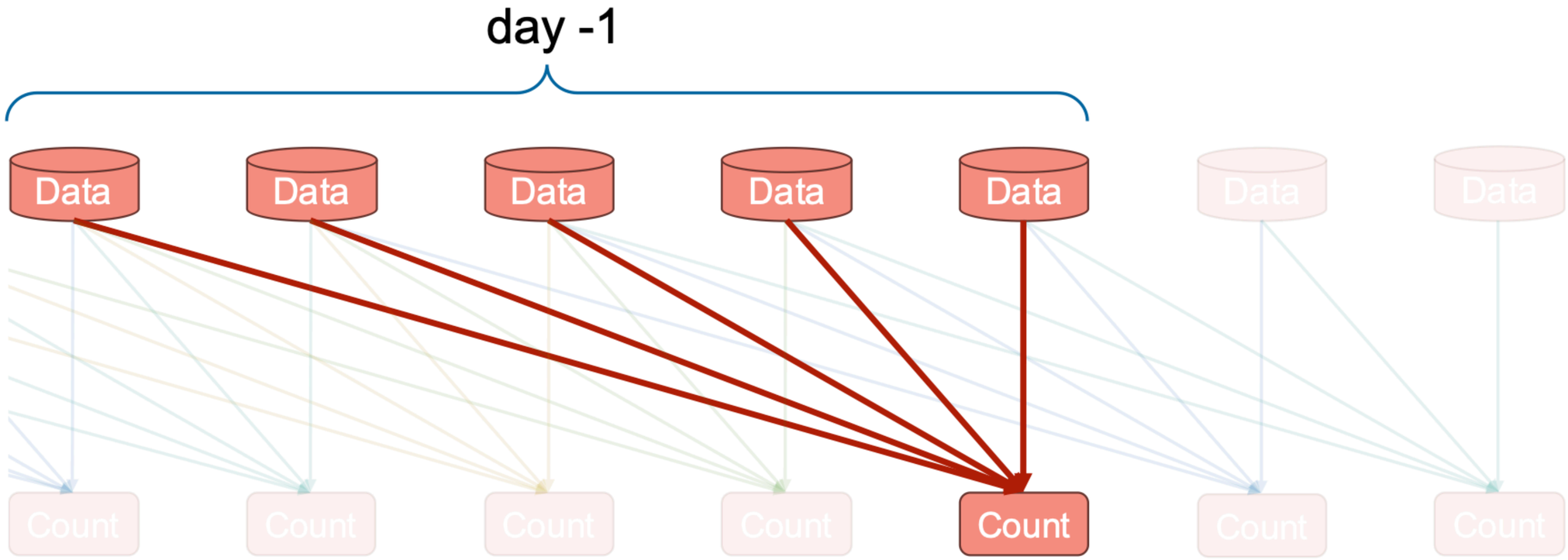
Data Sketches



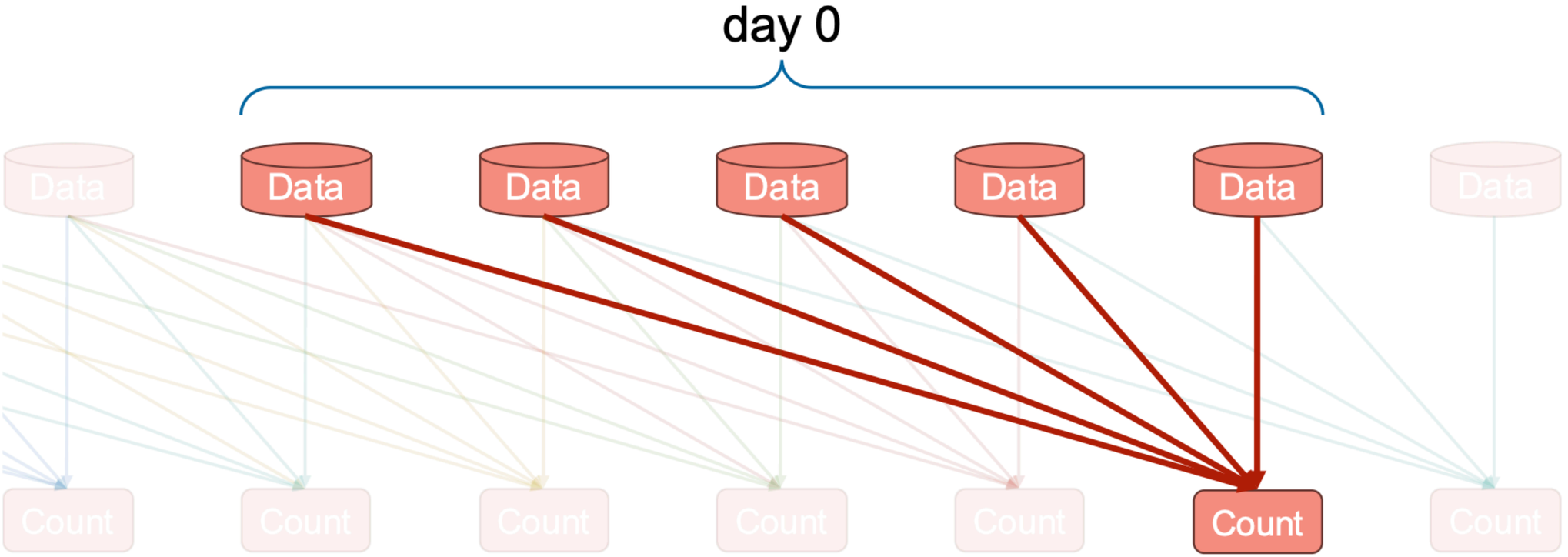
Data Sketches



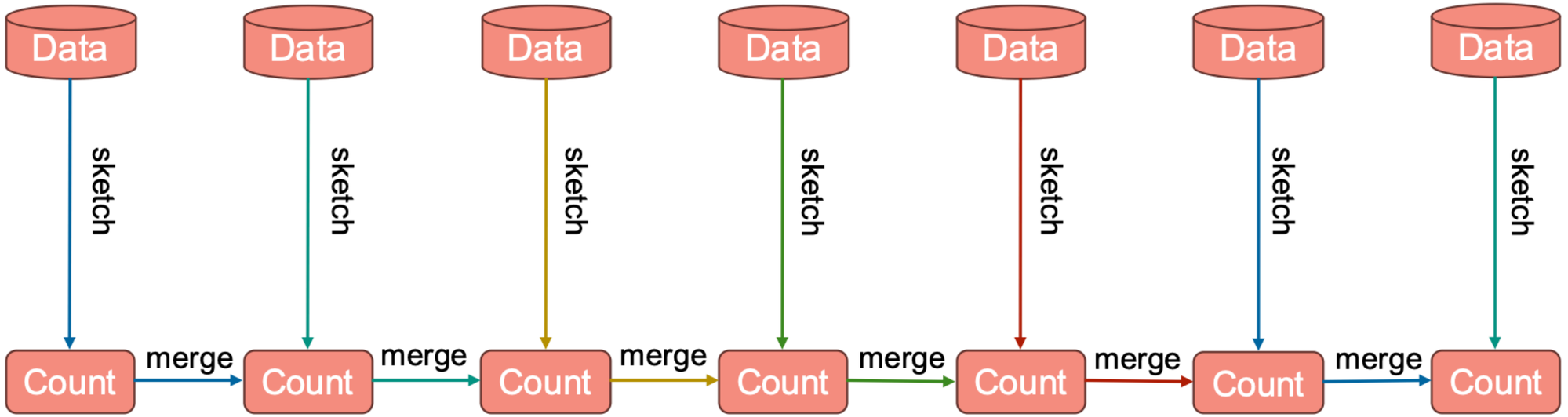
Data Sketches



Data Sketches

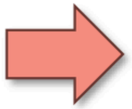


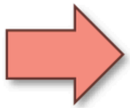
Data Sketches



LogLog

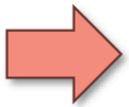
Алиса h(Алиса) 1110 1100

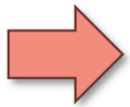




LogLog

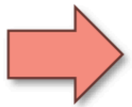
Алиса	h(Алиса)	1110 1100
Боб	h(Боб)	0110 1110

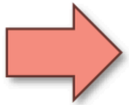




LogLog

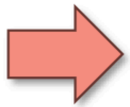
Алиса	h(Алиса)	1110 1100
Боб	h(Боб)	0110 1110
Алиса	h(Алиса)	1110 1100
Джек	h(Джек)	0111 0100
Боб	h(Боб)	0110 1110
Джон	h(Джон)	1111 0001
Мери	h(Мери)	1010 0010
Марта	h(Марта)	0010 1001
Джек	h(Джек)	0111 0100
Том	h(Том)	1110 0011
Мери	h(Мери)	1010 0010
Боб	h(Боб)	0110 1110

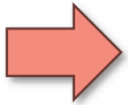


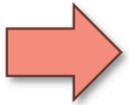


LogLog

Алиса	h(Алиса)	1110 1100
Боб	h(Боб)	0110 1110
Алиса	h(Алиса)	1110 1100
Джек	h(Джек)	0111 0100
Боб	h(Боб)	0110 1110
Джон	h(Джон)	1111 0001
Мери	h(Мери)	1010 0010
Марта	h(Марта)	0010 1001
Джек	h(Джек)	0111 0100
Том	h(Том)	1110 0011
Мери	h(Мери)	1010 0010
Боб	h(Боб)	0110 1110



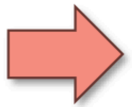


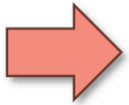


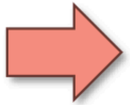
2^3 (8)

LogLog

Алиса	h(Алиса)	1110 1100
Боб	h(Боб)	0110 1110
Алиса	h(Алиса)	1110 1100
Джек	h(Джек)	0111 0100
Боб	h(Боб)	0110 1110
Джон	h(Джон)	1111 0001
Мери	h(Мери)	1010 0010
Марта	h(Марта)	0010 1001
Джек	h(Джек)	0111 0100
Том	h(Том)	1110 0011
Мери	h(Мери)	1010 0010
Боб	h(Боб)	0110 1110





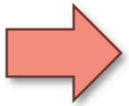


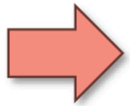
2^3 (8)

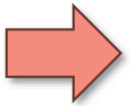
12 элементов
7 уникальных

LogLog

Алиса	h(Алиса)	1110 1100
Боб	h(Боб)	0110 1110
Алиса	h(Алиса)	1110 1100
Джек	h(Джек)	0111 0100
Боб	h(Боб)	0110 1110
Джон	h(Джон)	1111 0001
Мери	h(Мери)	1010 0010
Марта	h(Марта)	0000 1101
Джек	h(Джек)	0111 0100
Том	h(Том)	1110 0011
Мери	h(Мери)	1010 0010
Боб	h(Боб)	0110 1110



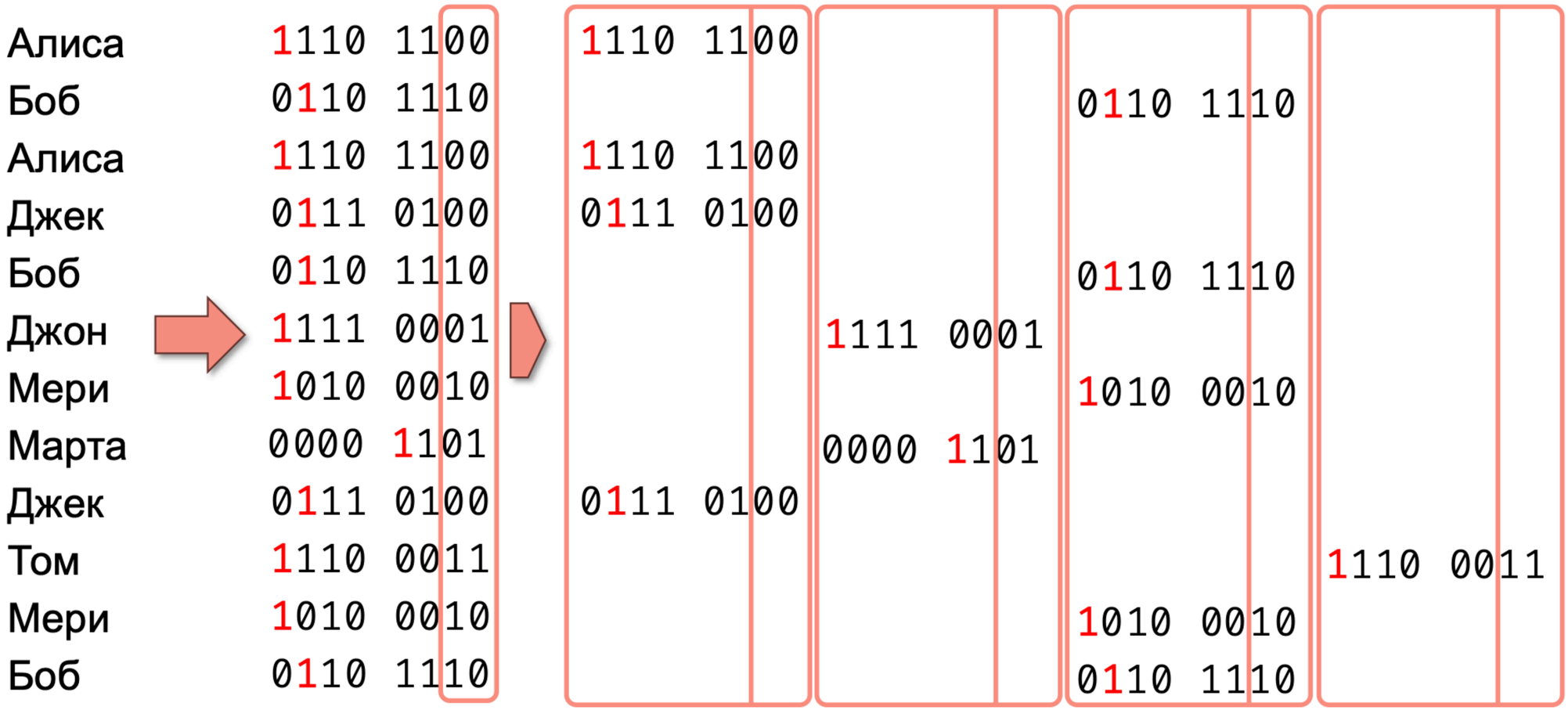




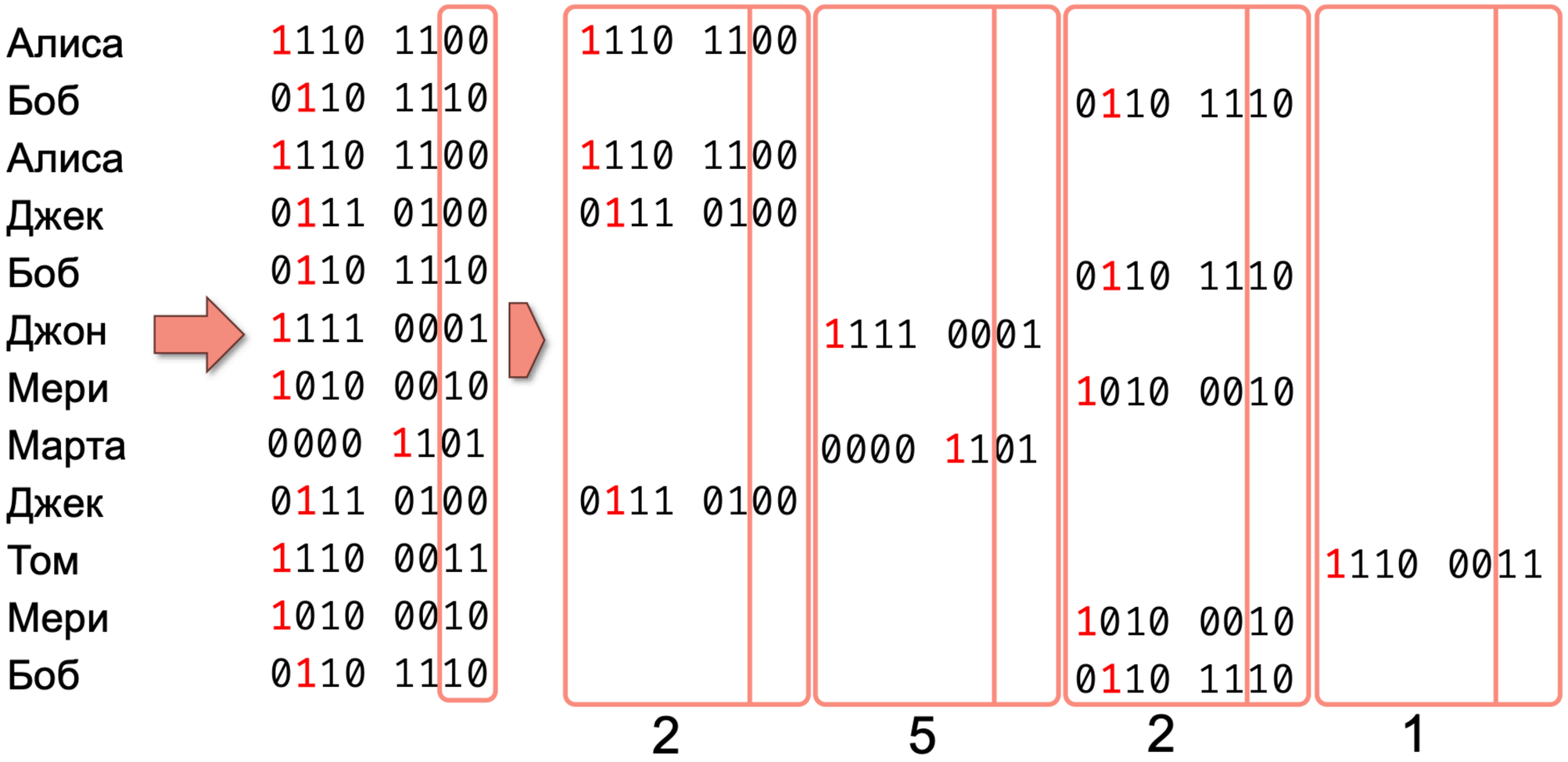
2^5 (32)

12 элементов
7 уникальных

LogLog

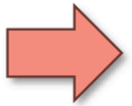

Алиса 1110 1100
Боб 0110 1110
Алиса 1110 1100
Джек 0111 0100
Боб 0110 1110
Джон 1111 0001
Мери 1010 0010
Марта 0000 1101
Джек 0111 0100
Том 1110 0011
Мери 1010 0010
Боб 0110 1110
1110 1100
0111 0100
1111 0001
0000 1101
0111 0100
0110 1110
0110 1110
1010 0010
1010 0010
0110 1110
1110 0011

LogLog

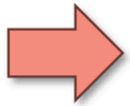

Алиса 1110 1100
Боб 0110 1110
Алиса 1110 1100
Джек 0111 0100
Боб 0110 1110
Джон 1111 0001
Мери 1010 0010
Марта 0000 1101
Джек 0111 0100
Том 1110 0011
Мери 1010 0010
Боб 0110 1110
1110 1100
1110 1100
0111 0100
0111 0100
0110 1110
0110 1110
1111 0001
1111 0001
1010 0010
1010 0010
0000 1101
0000 1101
1110 0011
1110 0011
2
5
2
1

LogLog

Алиса
Боб
Алиса
Джек
Боб
Джон
Мери
Марта
Джек
Том
Мери
Боб



$$4 * 2^{\frac{2+5+2+1}{4}}$$

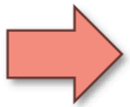


22,63

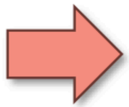
12 элементов
7 уникальных

LogLog

Алиса
Боб
Алиса
Джек
Боб
Джон
Мери
Марта
Джек
Том
Мери
Боб



$$4 * 2^{\frac{2+5+2+1}{4}}$$



22, 63

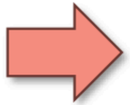
12 элементов
7 уникальных

было

2^5 (32)

LogLog

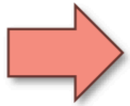
Алиса
Боб
Алиса
Джек
Боб
Джон
Мери
Марта
Джек
Том
Мери
Боб



$$4 * 2^{\frac{2+5+2+1}{4}} * \alpha$$

LogLog

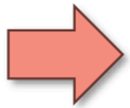
Алиса
Боб
Алиса
Джек
Боб
Джон
Мери
Марта
Джек
Том
Мери
Боб



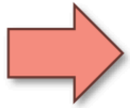
$$4 * 2^{\frac{2+5+2+1}{4}} * (0{,}39701 - \frac{2\pi^2 + (\ln 2)^2}{48 * 4})$$

LogLog

Алиса
Боб
Алиса
Джек
Боб
Джон
Мери
Марта
Джек
Том
Мери
Боб



$$4 * 2^{\frac{2+5+2+1}{4}} * (0{,}39701 - \frac{2\pi^2 + (\ln 2)^2}{48 * 4})$$



6, 6

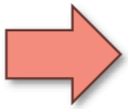
7 уникальных

было

22, 63

HyperLogLog

Алиса
Боб
Алиса
Джек
Боб
Джон
Мери
Марта
Джек
Том
Мери
Боб

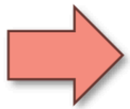


$$4 * \frac{4}{2^{-2} + 2^{-5} + 2^{-2} + 2^{-1}} * \alpha$$

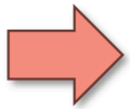
$$\alpha \sim \frac{1}{2 \ln 2 \left(1 + \frac{1}{4}(3 \ln 2 - 1) + 4^{-2}\right)}$$

HyperLogLog

Алиса
Боб
Алиса
Джек
Боб
Джон
Мери
Марта
Джек
Том
Мери
Боб



$$4 * \frac{4}{2^{-2} + 2^{-5} + 2^{-2} + 2^{-1}} * \alpha$$



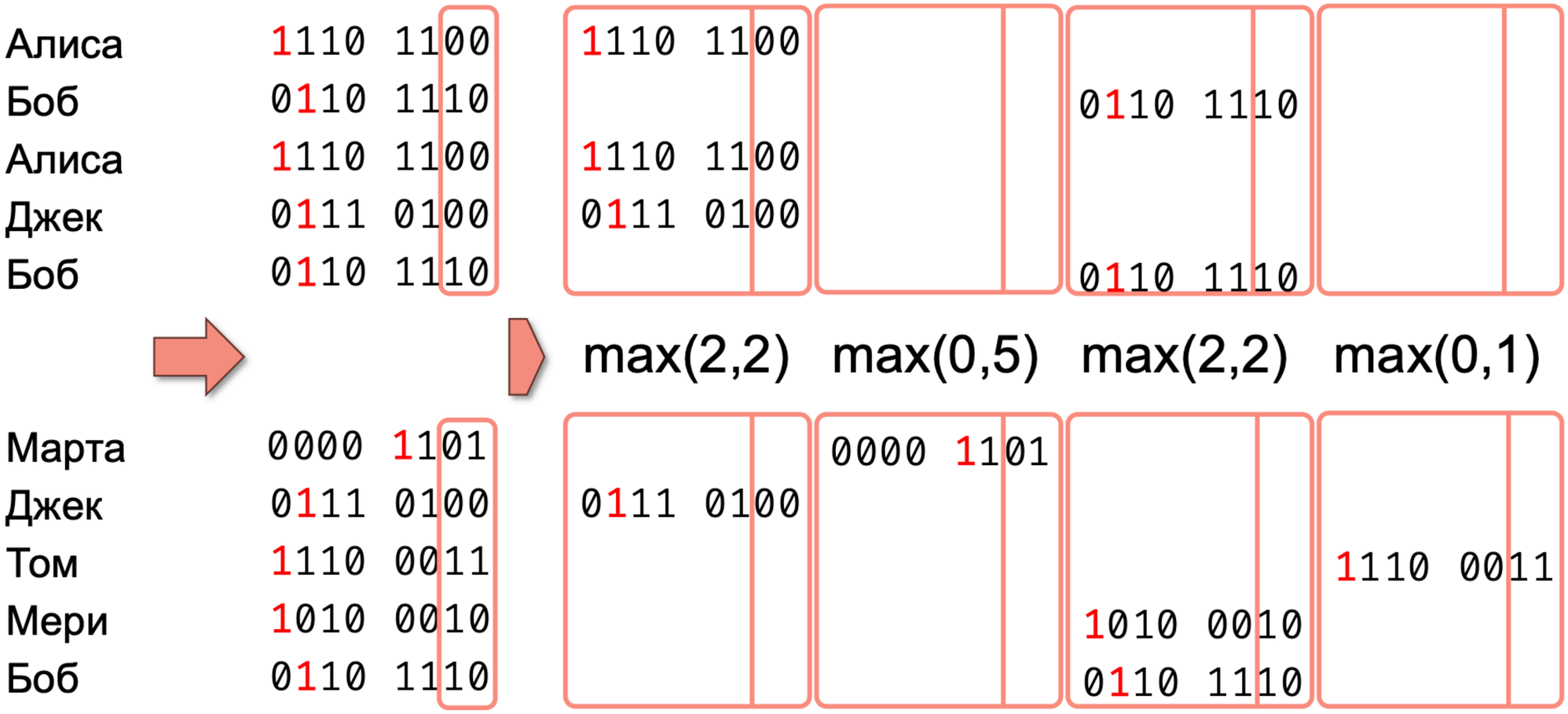
8, 6

7 уникальных

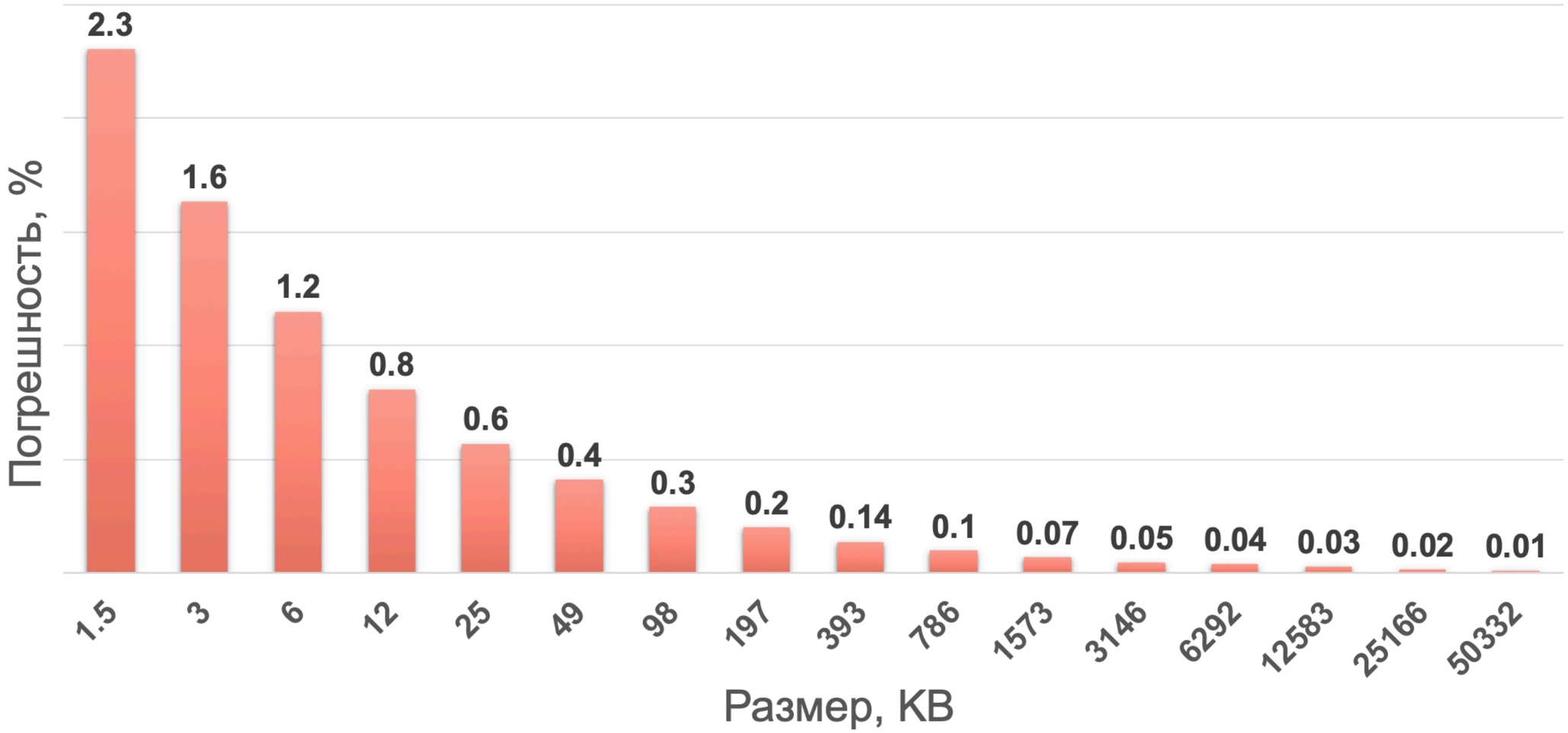
было

22, 63

HyperLogLog



Погрешность HyperLogLog



Погрешность HyperLogLog

10 KB за 1%

50 MB за 0,01%

для произвольного потока данных